Project 4

Soniya Shahi

2024-12-10

```
library(readxl)
library(GGally)
library(lme4)
library(lmerTest)
library(dplyr)
library(quantreg)
library(splines)
library(emmeans)
```

Loading the dataset from the excel file

```
data <- read_excel("DSL-StrongPasswordData.xlsx")
head(data)
```

```
## # A tibble: 6 x 34
##   subject sessionIndex   rep H.period DD.period.t UD.period.t    H.t DD.t.i
##   <chr>          <dbl> <dbl>    <dbl>       <dbl>       <dbl>  <dbl>  <dbl>
## 1 s002               1     1    0.149       0.398       0.249  0.107   0.167
## 2 s002               1     2    0.111       0.345       0.234  0.0694  0.128
## 3 s002               1     3    0.133       0.207       0.0744 0.0731  0.129
## 4 s002               1     4    0.129       0.252       0.122  0.106   0.250
## 5 s002               1     5    0.125       0.232       0.107  0.0895  0.168
## 6 s002               1     6    0.139       0.234       0.0949 0.0813  0.130
## # i 26 more variables: UD.t.i <dbl>, H.i <dbl>, DD.i.e <dbl>, UD.i.e <dbl>,
## #   H.e <dbl>, DD.e.five <dbl>, UD.e.five <dbl>, H.five <dbl>,
## #   DD.five.Shift.r <dbl>, UD.five.Shift.r <dbl>, H.Shift.r <dbl>,
## #   DD.Shift.r.o <dbl>, UD.Shift.r.o <dbl>, H.o <dbl>, DD.o.a <dbl>,
## #   UD.o.a <dbl>, H.a <dbl>, DD.a.n <dbl>, UD.a.n <dbl>, H.n <dbl>,
## #   DD.n.l <dbl>, UD.n.l <dbl>, H.l <dbl>, DD.l.Return <dbl>,
## #   UD.l.Return <dbl>, H.Return <dbl>
```

```
#checking the dimension of the data
dim(data)
```

```
## [1] 20400    34
```

There are 20400 rows and 34 columns.

```r
data$subject<-factor(data$subject)
data$rep <- factor(data$rep)
data$sessionIndex <- factor(data$sessionIndex)
str(data)
```

```
## tibble [20,400 x 34] (S3: tbl_df/tbl/data.frame)
##  $ subject        : Factor w/ 51 levels "s002","s003",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ sessionIndex   : Factor w/ 8 levels "1","2","3","4",..: 1 1 1 1 1 1 1 1 1 1 ...
##  $ rep            : Factor w/ 50 levels "1","2","3","4",..: 1 2 3 4 5 6 7 8 9 10 ...
##  $ H.period       : num [1:20400] 0.149 0.111 0.133 0.129 0.125 ...
##  $ DD.period.t    : num [1:20400] 0.398 0.345 0.207 0.252 0.232 ...
##  $ UD.period.t    : num [1:20400] 0.2488 0.234 0.0744 0.1224 0.1068 ...
##  $ H.t            : num [1:20400] 0.1069 0.0694 0.0731 0.1059 0.0895 ...
##  $ DD.t.i         : num [1:20400] 0.167 0.128 0.129 0.249 0.168 ...
##  $ UD.t.i         : num [1:20400] 0.0605 0.0589 0.056 0.1436 0.0781 ...
##  $ H.i            : num [1:20400] 0.1169 0.0908 0.0821 0.104 0.0903 ...
##  $ DD.i.e         : num [1:20400] 0.221 0.136 0.154 0.204 0.159 ...
##  $ UD.i.e         : num [1:20400] 0.1043 0.0449 0.0721 0.0998 0.0686 ...
##  $ H.e            : num [1:20400] 0.1417 0.0829 0.0808 0.09 0.0805 ...
##  $ DD.e.five      : num [1:20400] 1.188 1.197 1.041 1.056 0.863 ...
##  $ UD.e.five      : num [1:20400] 1.047 1.114 0.96 0.966 0.782 ...
##  $ H.five         : num [1:20400] 0.1146 0.0689 0.0892 0.0913 0.0742 ...
##  $ DD.five.Shift.r: num [1:20400] 1.605 0.782 0.62 1.256 0.895 ...
##  $ UD.five.Shift.r: num [1:20400] 1.491 0.713 0.531 1.165 0.821 ...
##  $ H.Shift.r      : num [1:20400] 0.107 0.157 0.145 0.145 0.124 ...
##  $ DD.Shift.r.o   : num [1:20400] 0.759 0.788 0.72 0.755 0.763 ...
##  $ UD.Shift.r.o   : num [1:20400] 0.652 0.631 0.574 0.61 0.639 ...
##  $ H.o            : num [1:20400] 0.1016 0.1066 0.1365 0.0956 0.043 ...
##  $ DD.o.a         : num [1:20400] 0.214 0.168 0.293 0.153 0.198 ...
##  $ UD.o.a         : num [1:20400] 0.112 0.0618 0.1566 0.0574 0.1545 ...
##  $ H.a            : num [1:20400] 0.135 0.141 0.162 0.146 0.131 ...
##  $ DD.a.n         : num [1:20400] 0.148 0.256 0.233 0.163 0.158 ...
##  $ UD.a.n         : num [1:20400] 0.0135 0.1146 0.0711 0.0172 0.027 ...
##  $ H.n            : num [1:20400] 0.0932 0.1146 0.1172 0.0866 0.0884 ...
##  $ DD.n.l         : num [1:20400] 0.351 0.264 0.271 0.234 0.252 ...
##  $ UD.n.l         : num [1:20400] 0.258 0.15 0.153 0.147 0.163 ...
##  $ H.l            : num [1:20400] 0.1338 0.0839 0.1085 0.0845 0.0903 ...
##  $ DD.l.Return    : num [1:20400] 0.351 0.276 0.285 0.323 0.252 ...
##  $ UD.l.Return    : num [1:20400] 0.217 0.192 0.176 0.239 0.161 ...
##  $ H.Return       : num [1:20400] 0.0742 0.0747 0.0945 0.0813 0.0818 0.0591 0.089 0.0679 0.0634 0.08
```

Extracting unique values from 'subject', 'rep', and 'sessionIndex' columns

```r
unique(data$subject)
```

```
##  [1] s002 s003 s004 s005 s007 s008 s010 s011 s012 s013 s015 s016 s017 s018 s019
## [16] s020 s021 s022 s024 s025 s026 s027 s028 s029 s030 s031 s032 s033 s034 s035
## [31] s036 s037 s038 s039 s040 s041 s042 s043 s044 s046 s047 s048 s049 s050 s051
## [46] s052 s053 s054 s055 s056 s057
## 51 Levels: s002 s003 s004 s005 s007 s008 s010 s011 s012 s013 s015 s016 ... s057
```

```r
unique(data$rep)
```

```
##  [1] 1  2  3  4  5  6  7  8  9  10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
## 50 Levels: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 ... 50
```

```r
unique(data$sessionIndex)
```

```
## [1] 1 2 3 4 5 6 7 8
## Levels: 1 2 3 4 5 6 7 8
```

Sub Task 1

Exploratory Data Analysis

Creating summarized columns for Total typing time, total key holding time, total key up and key down time and total key down down time as 'total_time', 'ud_time', 'dd_time' amd hold_time respectively.

```r
data$total_time <- rowSums(data[, !(colnames(data) %in% c("subject", "sessionIndex", "rep"))])
# Sum columns that start with "H" row-wise
data$hold_time <- apply(data[, grepl("^H", colnames(data))], 1, sum)

# Sum columns that start with "U" row-wise
data$ud_time <- apply(data[, grepl("^U", colnames(data))], 1, sum)

# Sum columns that start with "D" row-wise
data$dd_time <- apply(data[, grepl("^D", colnames(data))], 1, sum)
data
```

```
## # A tibble: 20,400 x 38
##    subject sessionIndex rep   H.period DD.period.t UD.period.t    H.t DD.t.i
##    <fct>   <fct>        <fct>    <dbl>       <dbl>       <dbl>  <dbl>  <dbl>
##  1 s002    1            1        0.149       0.398       0.249  0.107   0.167
##  2 s002    1            2        0.111       0.345       0.234  0.0694  0.128
##  3 s002    1            3        0.133       0.207       0.0744 0.0731  0.129
##  4 s002    1            4        0.129       0.252       0.122  0.106   0.250
##  5 s002    1            5        0.125       0.232       0.107  0.0895  0.168
##  6 s002    1            6        0.139       0.234       0.0949 0.0813  0.130
##  7 s002    1            7        0.106       0.207       0.100  0.0866  0.137
##  8 s002    1            8        0.0929      0.181       0.0881 0.0818  0.138
##  9 s002    1            9        0.0966      0.180       0.0831 0.0771  0.130
## 10 s002    1            10       0.109       0.181       0.0714 0.0731  0.146
## # i 20,390 more rows
## # i 30 more variables: UD.t.i <dbl>, H.i <dbl>, DD.i.e <dbl>, UD.i.e <dbl>,
## #   H.e <dbl>, DD.e.five <dbl>, UD.e.five <dbl>, H.five <dbl>,
## #   DD.five.Shift.r <dbl>, UD.five.Shift.r <dbl>, H.Shift.r <dbl>,
## #   DD.Shift.r.o <dbl>, UD.Shift.r.o <dbl>, H.o <dbl>, DD.o.a <dbl>,
## #   UD.o.a <dbl>, H.a <dbl>, DD.a.n <dbl>, UD.a.n <dbl>, H.n <dbl>,
## #   DD.n.l <dbl>, UD.n.l <dbl>, H.l <dbl>, DD.l.Return <dbl>, ...
```

Creating summary statistics for all the summarized variables for each sessions.

```r
# Using dplyr for total_time
data_grouped_total <- data %>%
  group_by(sessionIndex) %>%
  summarize(
    count = n(),
    mean = mean(total_time),
    std = sd(total_time),
    min = min(total_time),
    q25 = quantile(total_time, 0.25),
    median = quantile(total_time, 0.5),
    q75 = quantile(total_time, 0.75),
    max = max(total_time)
  )

# Print header and summary statistics for total_time
cat("Summary Statistics for total_time\n")
```

```
## Summary Statistics for total_time
```

```r
print(data_grouped_total)
```

```
## # A tibble: 8 x 9
##   sessionIndex count  mean   std   min   q25 median   q75   max
##   <fct>        <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1 1             2550  6.43  3.53  2.47  4.45   5.52  7.14  71.9
## 2 2             2550  5.54  2.22  2.65  4.00   4.90  6.30  20.1
## 3 3             2550  5.29  2.24  2.60  3.80   4.69  6.00  34.4
## 4 4             2550  4.96  2.09  2.48  3.60   4.37  5.59  21.6
## 5 5             2550  4.78  1.85  2.37  3.50   4.34  5.37  17.5
## 6 6             2550  4.61  1.67  2.28  3.41   4.19  5.33  14.5
## 7 7             2550  4.49  1.59  2.22  3.38   4.07  5.18  19.3
## 8 8             2550  4.48  1.78  2.11  3.29   4.07  5.17  19.4
```

```r
# Another option using base R
cat("\nSummary Statistics for total_time using base R\n")
```

```
##
## Summary Statistics for total_time using base R
```

```r
summary(data$total_time ~ data$sessionIndex)
```

```
##  Length   Class    Mode
##       3 formula    call
```

```r
# Using dplyr for hold_time
data_grouped_hold <- data %>%
  group_by(sessionIndex) %>%
  summarize(
    count = n(),
    mean = mean(hold_time, na.rm = TRUE),
```

```
    std = sd(hold_time, na.rm = TRUE),
    min = min(hold_time, na.rm = TRUE),
    q25 = quantile(hold_time, 0.25, na.rm = TRUE),
    median = quantile(hold_time, 0.5, na.rm = TRUE),
    q75 = quantile(hold_time, 0.75, na.rm = TRUE),
    max = max(hold_time, na.rm = TRUE)
  )

# Print header and summary statistics for hold_time
cat("\nSummary Statistics for hold_time\n")
```

```
##
## Summary Statistics for hold_time
```

```
print(data_grouped_hold)
```

```
## # A tibble: 8 x 9
##   sessionIndex count  mean   std   min   q25 median   q75   max
##   <fct>        <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1 1             2550 0.953 0.216 0.348 0.817  0.933  1.08  2.07
## 2 2             2550 0.984 0.224 0.465 0.855  0.980  1.11  2.03
## 3 3             2550 0.978 0.236 0.430 0.843  0.965  1.09  3.27
## 4 4             2550 0.981 0.220 0.436 0.845  0.969  1.09  1.91
## 5 5             2550 0.984 0.211 0.478 0.852  0.975  1.09  1.86
## 6 6             2550 1.01  0.233 0.440 0.862  0.998  1.14  2.01
## 7 7             2550 1.01  0.237 0.438 0.870  0.990  1.14  2.04
## 8 8             2550 1.02  0.256 0.502 0.866  0.991  1.14  1.89
```

```
# Another option using base R
cat("\nSummary Statistics for hold_time using base R\n")
```

```
##
## Summary Statistics for hold_time using base R
```

```
summary(data$hold_time ~ data$sessionIndex)
```

```
##  Length   Class    Mode
##       3 formula    call
```

```
# Using dplyr for ud_time
data_grouped_ud <- data %>%
  group_by(sessionIndex) %>%
  summarize(
    count = n(),
    mean = mean(ud_time, na.rm = TRUE),
    std = sd(ud_time, na.rm = TRUE),
    min = min(ud_time, na.rm = TRUE),
    q25 = quantile(ud_time, 0.25, na.rm = TRUE),
    median = quantile(ud_time, 0.5, na.rm = TRUE),
    q75 = quantile(ud_time, 0.75, na.rm = TRUE),
```

```
    max = max(ud_time, na.rm = TRUE)
  )

# Print header and summary statistics for ud_time
cat("\nSummary Statistics for ud_time\n")
```

```
##
## Summary Statistics for ud_time
```

```
print(data_grouped_ud)
```

```
## # A tibble: 8 x 9
##   sessionIndex count  mean   std     min   q25 median   q75   max
##   <fct>        <int> <dbl> <dbl>   <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1 1             2550  2.30 1.79   0.320  1.32    1.83  2.65 34.9
## 2 2             2550  1.83 1.14   0.269  1.06    1.52  2.19  8.97
## 3 3             2550  1.71 1.15   0.208  0.952   1.42  2.05 16.5
## 4 4             2550  1.54 1.07   0.172  0.862   1.25  1.83 10.2
## 5 5             2550  1.45 0.962  0.129  0.787   1.23  1.76  7.83
## 6 6             2550  1.34 0.866 -0.0324 0.736   1.13  1.69  6.21
## 7 7             2550  1.27 0.836 -0.219  0.702   1.06  1.64  8.59
## 8 8             2550  1.26 0.937 -0.166  0.675   1.07  1.61  9.16
```

```
# Another option using base R
cat("\nSummary Statistics for ud_time using base R\n")
```

```
##
## Summary Statistics for ud_time using base R
```

```
summary(data$ud_time ~ data$sessionIndex)
```

```
##  Length   Class    Mode
##       3 formula    call
```

```
# Using dplyr for dd_time
data_grouped_dd <- data %>%
  group_by(sessionIndex) %>%
  summarize(
    count = n(),
    mean = mean(dd_time, na.rm = TRUE),
    std = sd(dd_time, na.rm = TRUE),
    min = min(dd_time, na.rm = TRUE),
    q25 = quantile(dd_time, 0.25, na.rm = TRUE),
    median = quantile(dd_time, 0.5, na.rm = TRUE),
    q75 = quantile(dd_time, 0.75, na.rm = TRUE),
    max = max(dd_time, na.rm = TRUE)
  )

# Print header and summary statistics for dd_time
cat("\nSummary Statistics for dd_time\n")
```

```
##
## Summary Statistics for dd_time
```

```
print(data_grouped_dd)
```

```
## # A tibble: 8 x 9
##   sessionIndex count  mean   std   min   q25 median   q75   max
##   <fct>        <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1 1             2550  3.17 1.77   1.18  2.18   2.71  3.53 35.9
## 2 2             2550  2.72 1.11   1.27  1.95   2.40  3.11 10.0
## 3 3             2550  2.60 1.12   1.24  1.86   2.30  2.96 17.2
## 4 4             2550  2.44 1.05   1.20  1.76   2.15  2.74 10.8
## 5 5             2550  2.35 0.927  1.14  1.71   2.12  2.64  8.72
## 6 6             2550  2.26 0.836  1.07  1.66   2.05  2.62  7.17
## 7 7             2550  2.20 0.796  1.04  1.65   1.99  2.55  9.61
## 8 8             2550  2.20 0.891  1.02  1.60   1.99  2.54  9.68
```

```
# Another option using base R
cat("\nSummary Statistics for dd_time using base R\n")
```

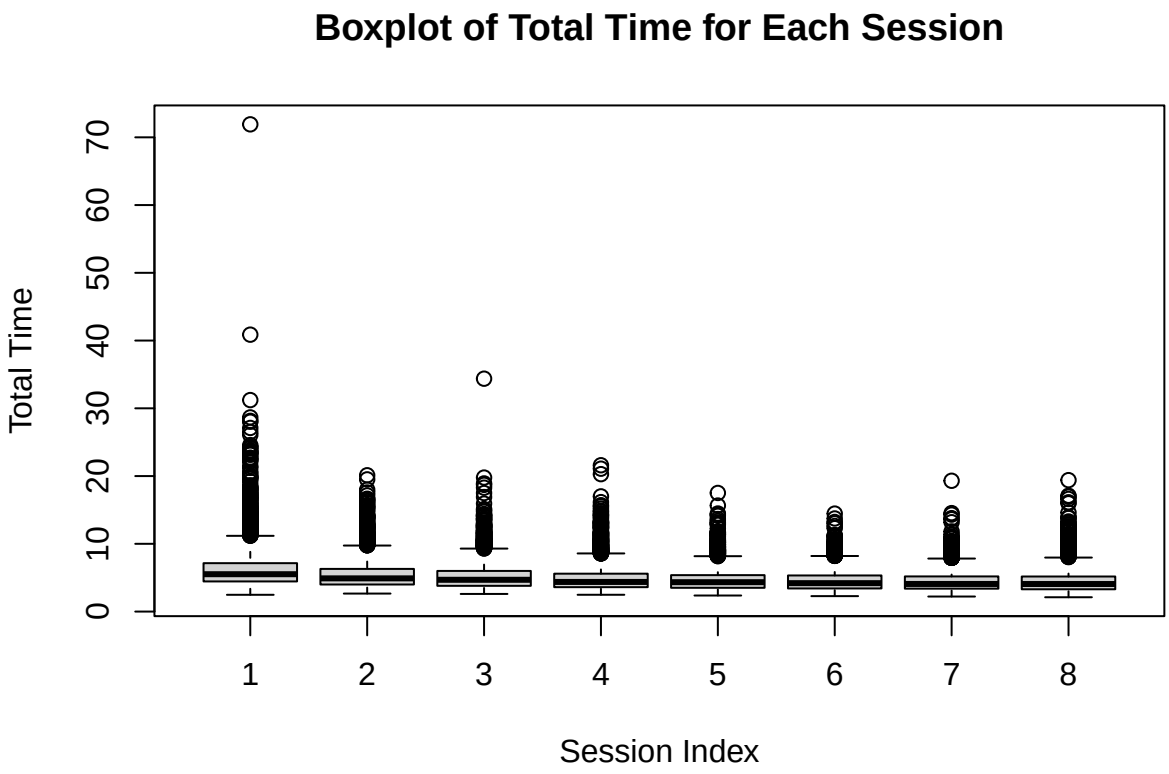
```
##
## Summary Statistics for dd_time using base R
```

```
summary(data$dd_time ~ data$sessionIndex)
```

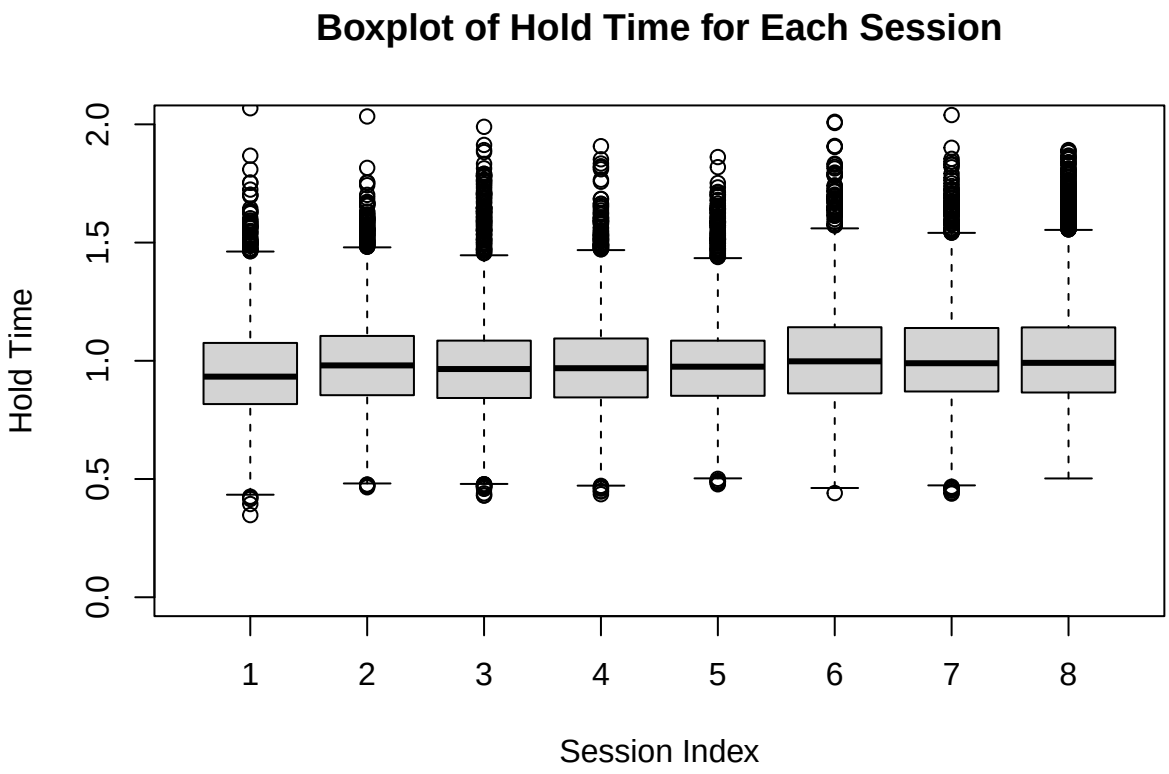
```
##  Length   Class    Mode
##       3 formula    call
```

Visualizing the summary statistics with the box plot.

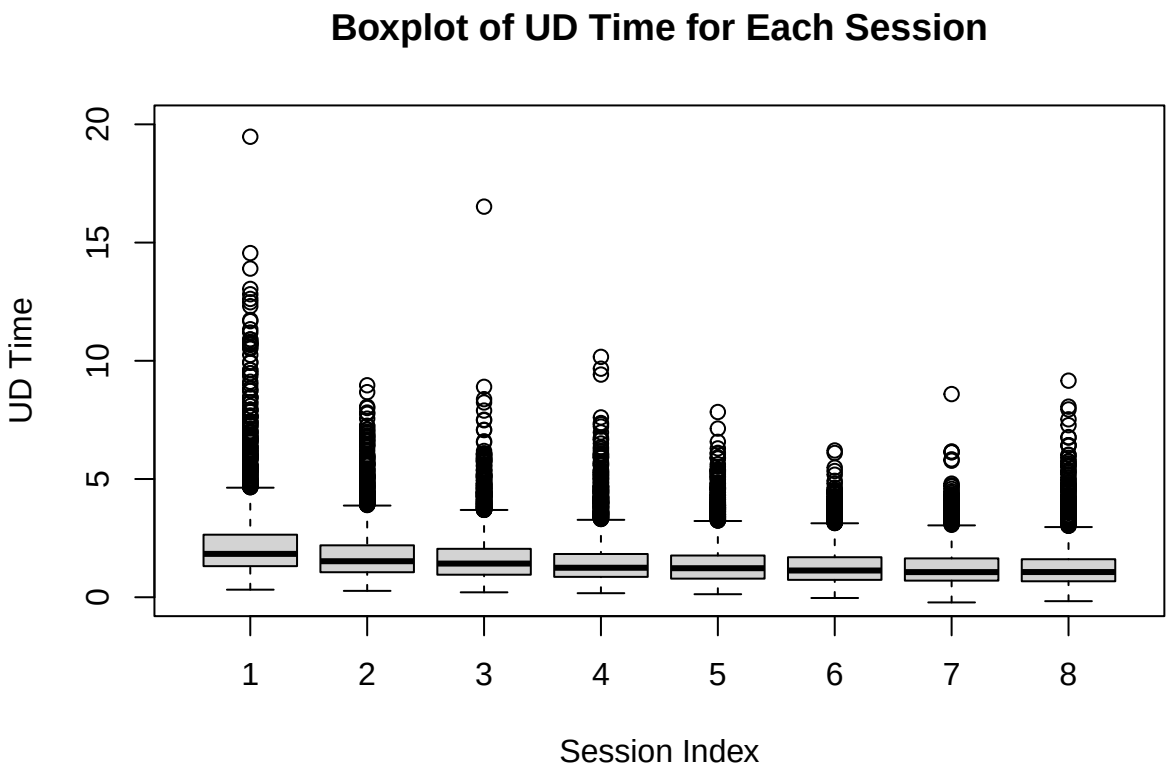
```
boxplot(total_time ~ sessionIndex, data = data,
        main = "Boxplot of Total Time for Each Session",
        xlab = "Session Index", ylab = "Total Time")
```



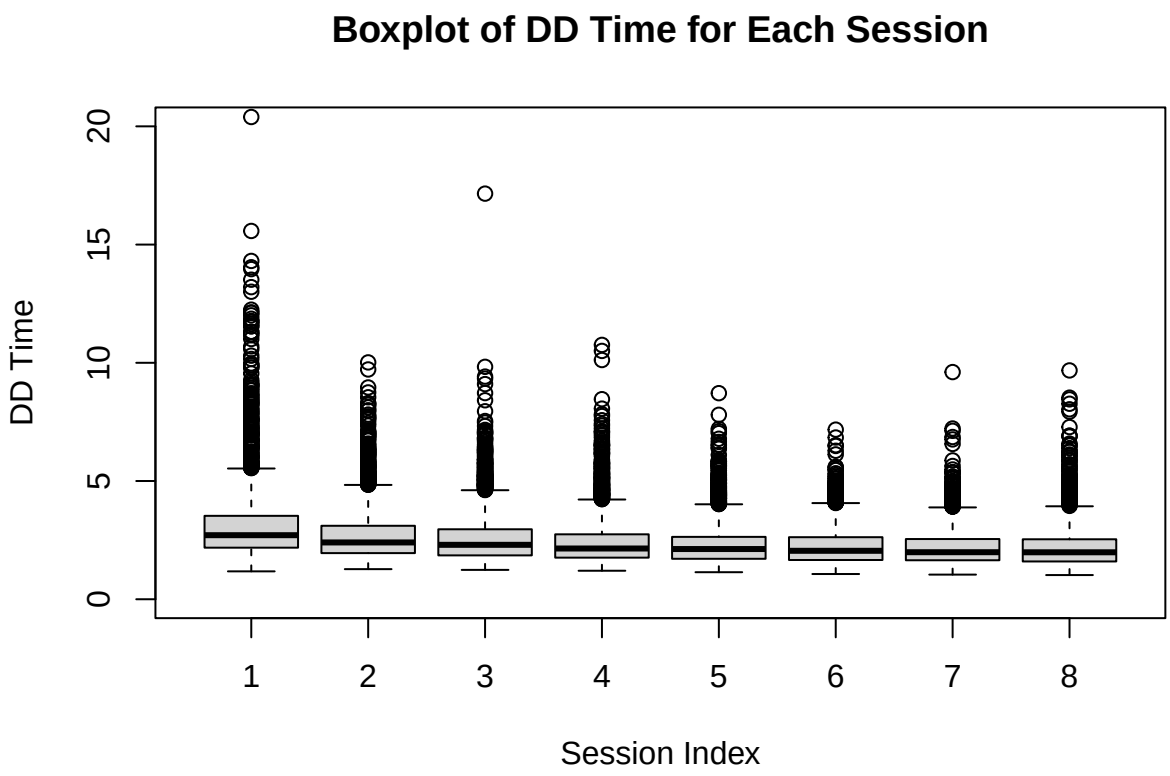

```r
# Boxplot of Hold Time for Each Session
boxplot(hold_time ~ sessionIndex, data = data,
        ylim = c(0, 2),
        main = "Boxplot of Hold Time for Each Session",
        xlab = "Session Index", ylab = "Hold Time")
```




```r
# Boxplot of UD Time for Each Session
boxplot(ud_time ~ sessionIndex, data = data,
        ylim = c(0, 20),
        main = "Boxplot of UD Time for Each Session",
        xlab = "Session Index", ylab = "UD Time")
```

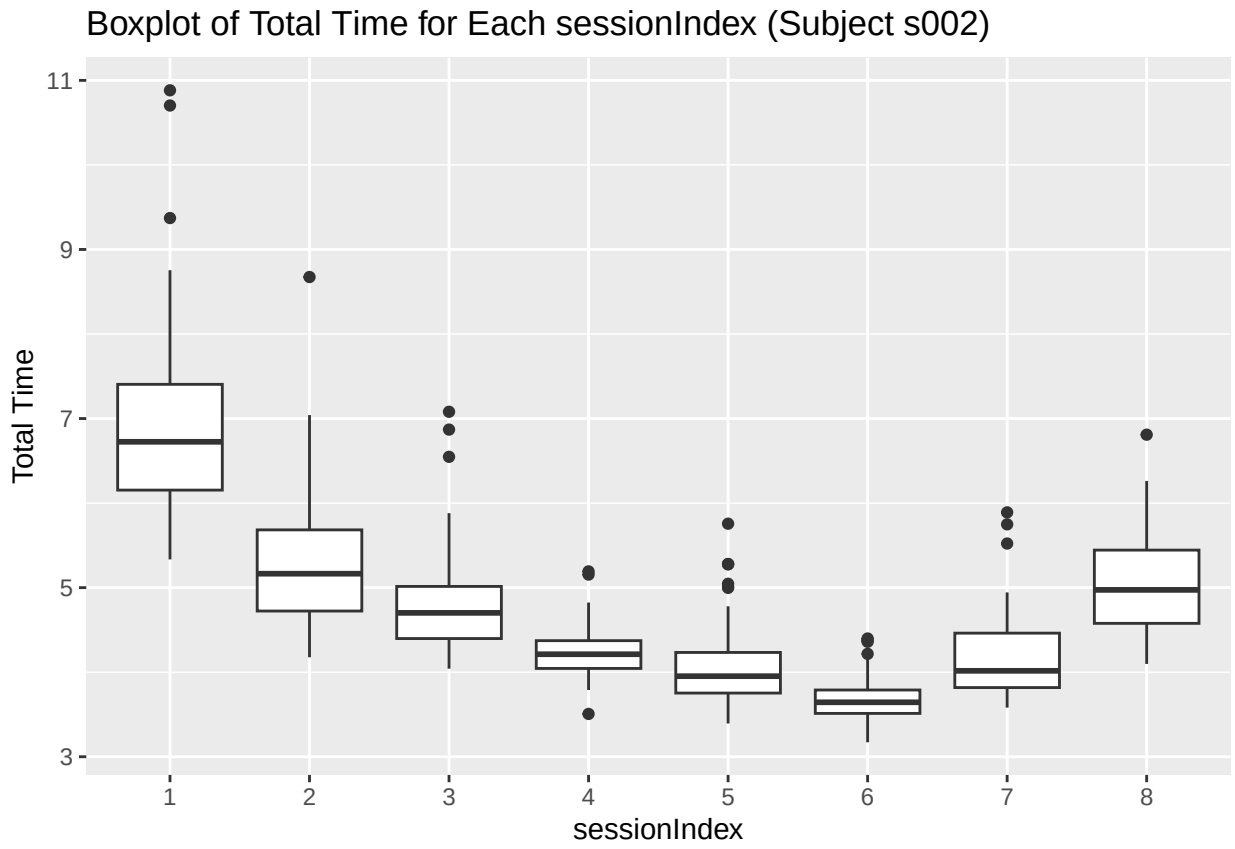



```
# Boxplot of DD Time for Each Session
boxplot(dd_time ~ sessionIndex, data = data,
        ylim = c(0, 20),
        main = "Boxplot of DD Time for Each Session",
        xlab = "Session Index", ylab = "DD Time")
```

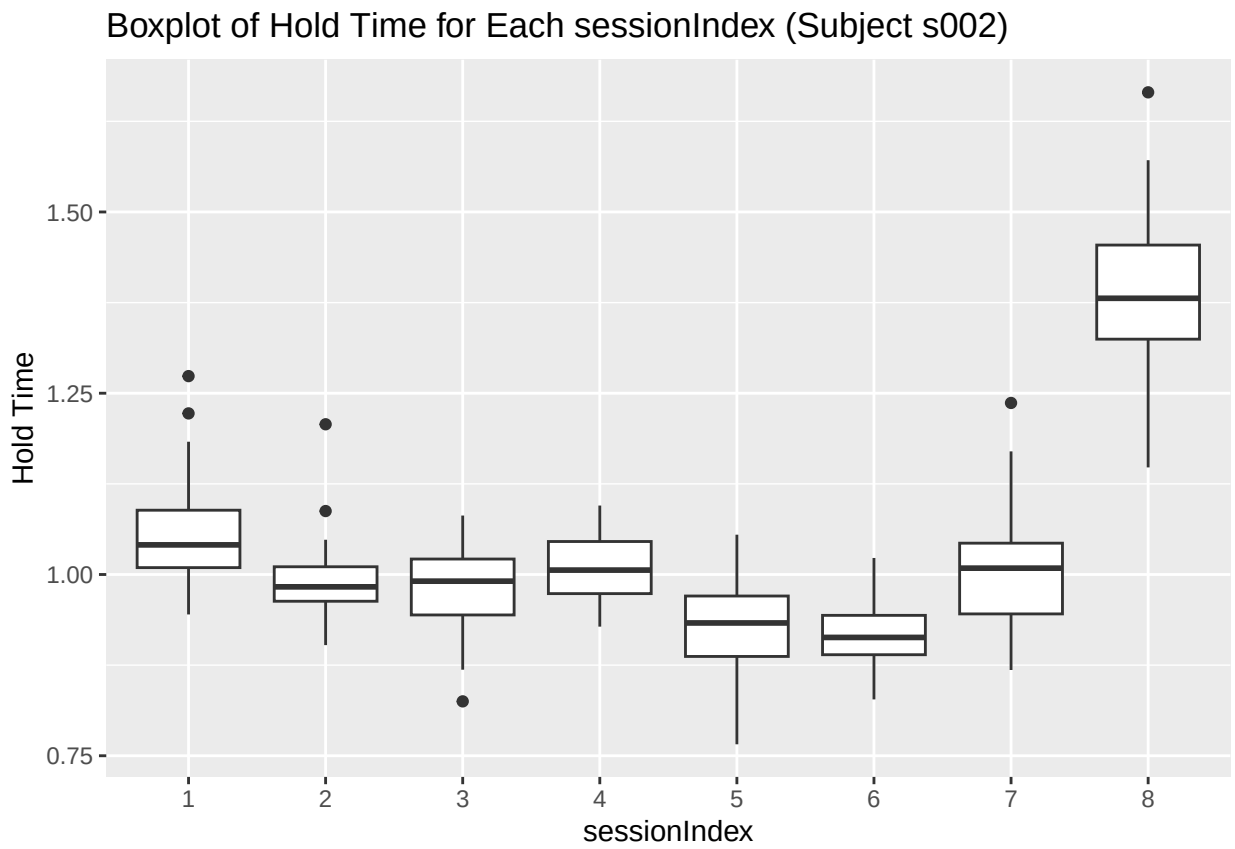


Visualizing the summary statistics for randomly selected subjects s002 and s010

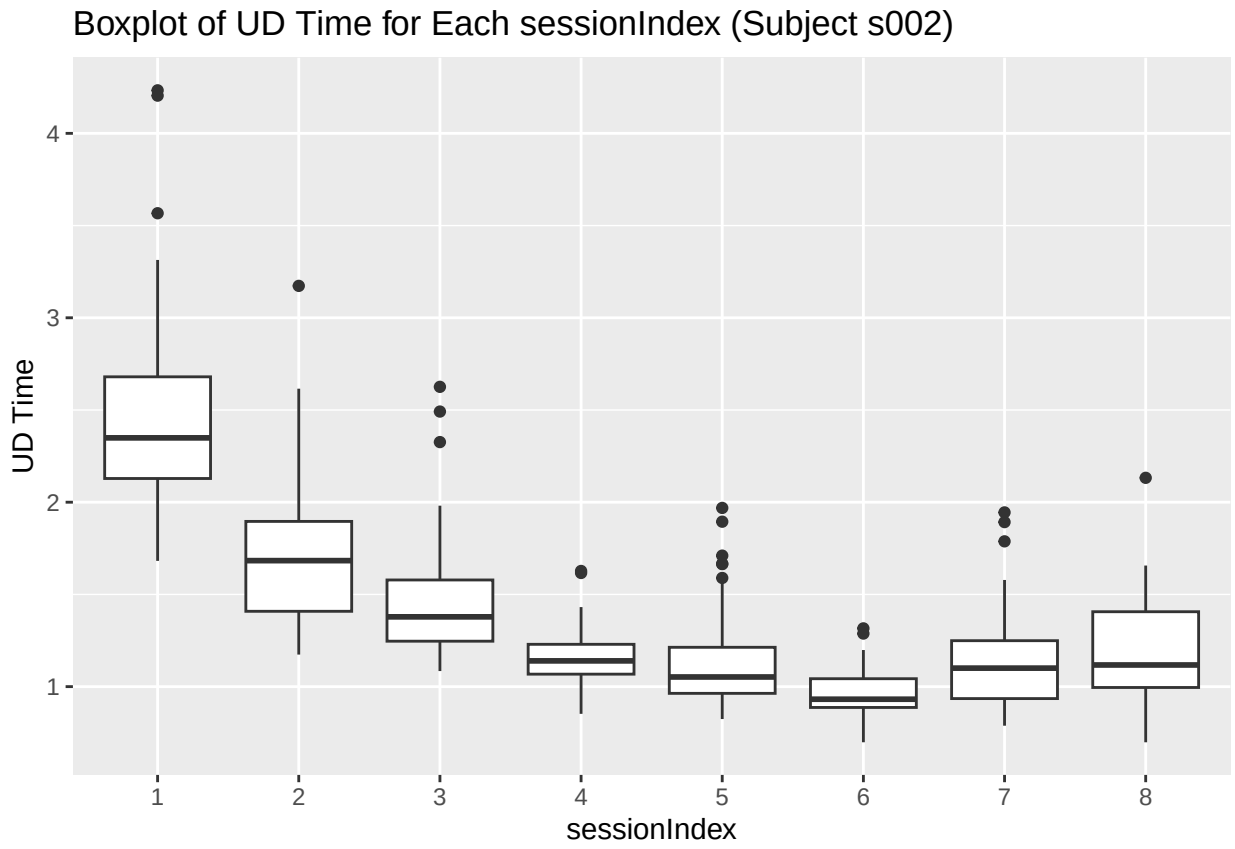
```
# Subset data for subject 's002'
subset_data <- data[data$subject == "s002", ]

# Create a box plot for 'total_time' for each 'sessionIndex'
ggplot(subset_data, aes(x = factor(sessionIndex), y = total_time)) +
  geom_boxplot() +
  labs(title = "Boxplot of Total Time for Each sessionIndex (Subject s002)",
       x = "sessionIndex", y = "Total Time")
```



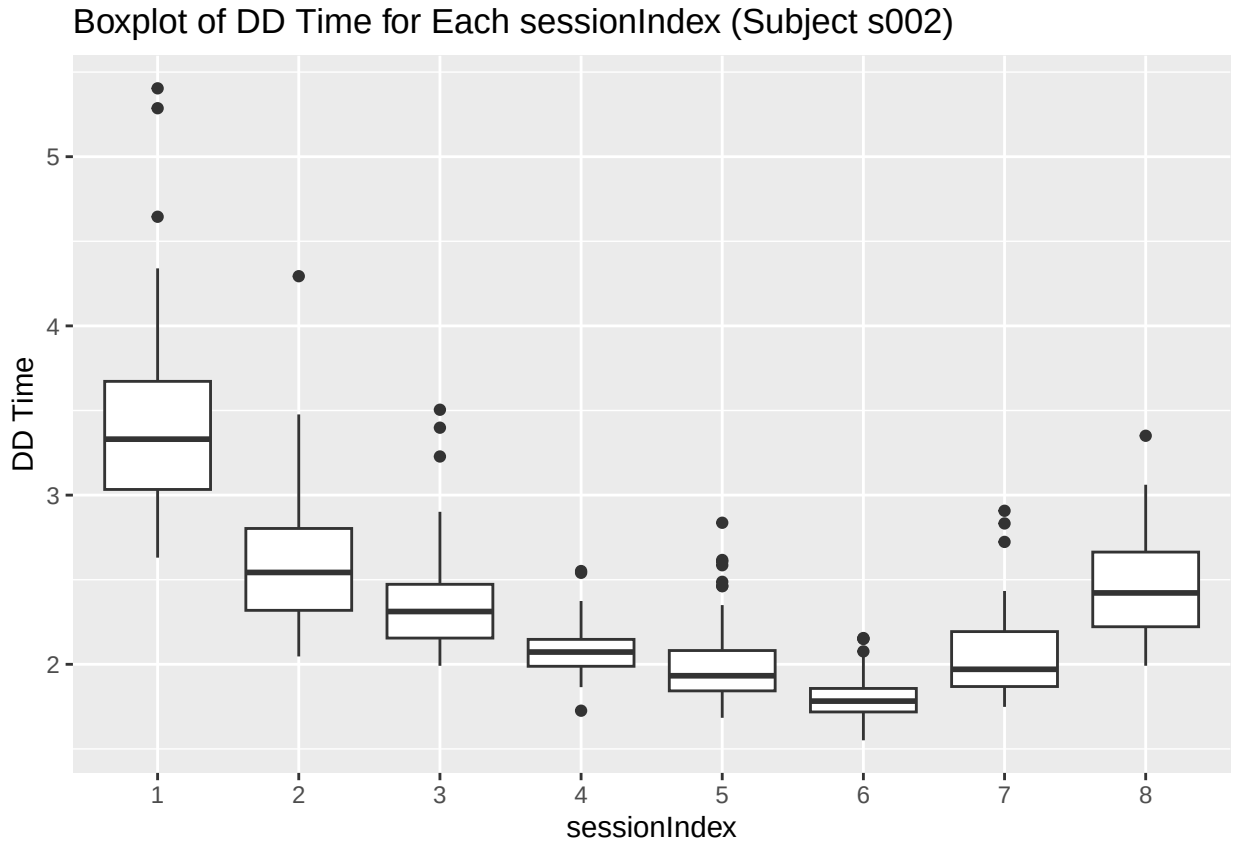

```
# Create a box plot for 'hold_time' for each 'sessionIndex'
ggplot(subset_data, aes(x = factor(sessionIndex), y = hold_time)) +
  geom_boxplot() +
  labs(title = "Boxplot of Hold Time for Each sessionIndex (Subject s002)",
       x = "sessionIndex", y = "Hold Time")
```



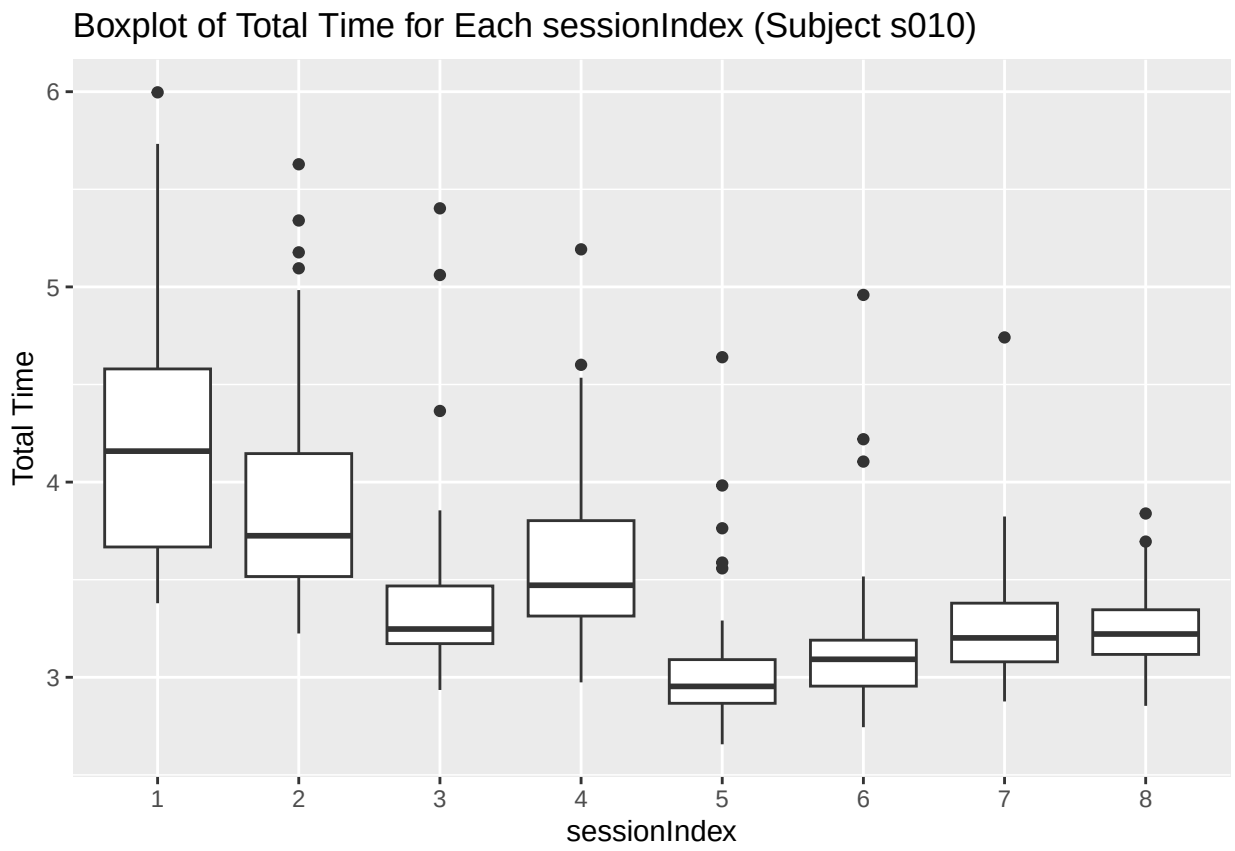

```r
# Create a box plot for 'ud_time' for each 'sessionIndex'
ggplot(subset_data, aes(x = factor(sessionIndex), y = ud_time)) +
  geom_boxplot() +
  labs(title = "Boxplot of UD Time for Each sessionIndex (Subject s002)",
       x = "sessionIndex", y = "UD Time")
```



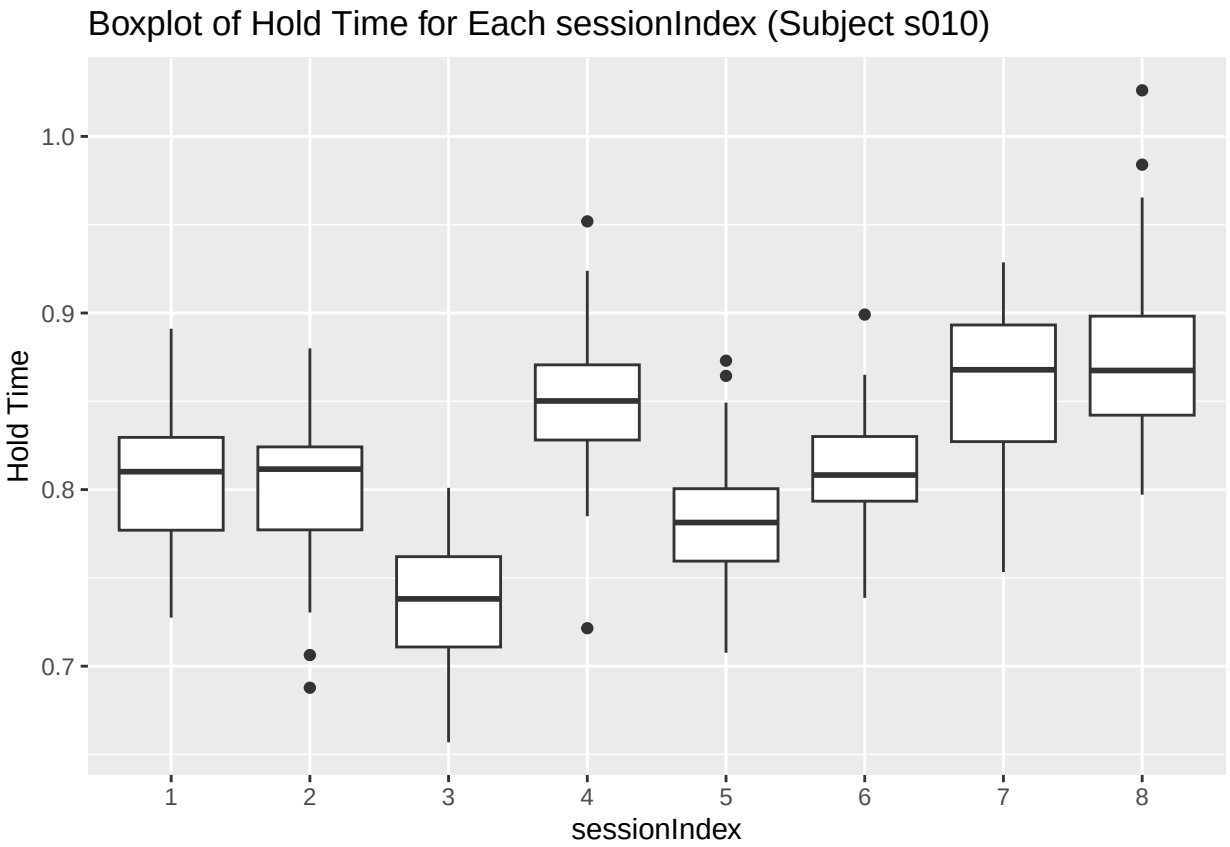

```
# Create a box plot for 'dd_time' for each 'sessionIndex'
ggplot(subset_data, aes(x = factor(sessionIndex), y = dd_time)) +
  geom_boxplot() +
  labs(title = "Boxplot of DD Time for Each sessionIndex (Subject s002)",
       x = "sessionIndex", y = "DD Time")
```



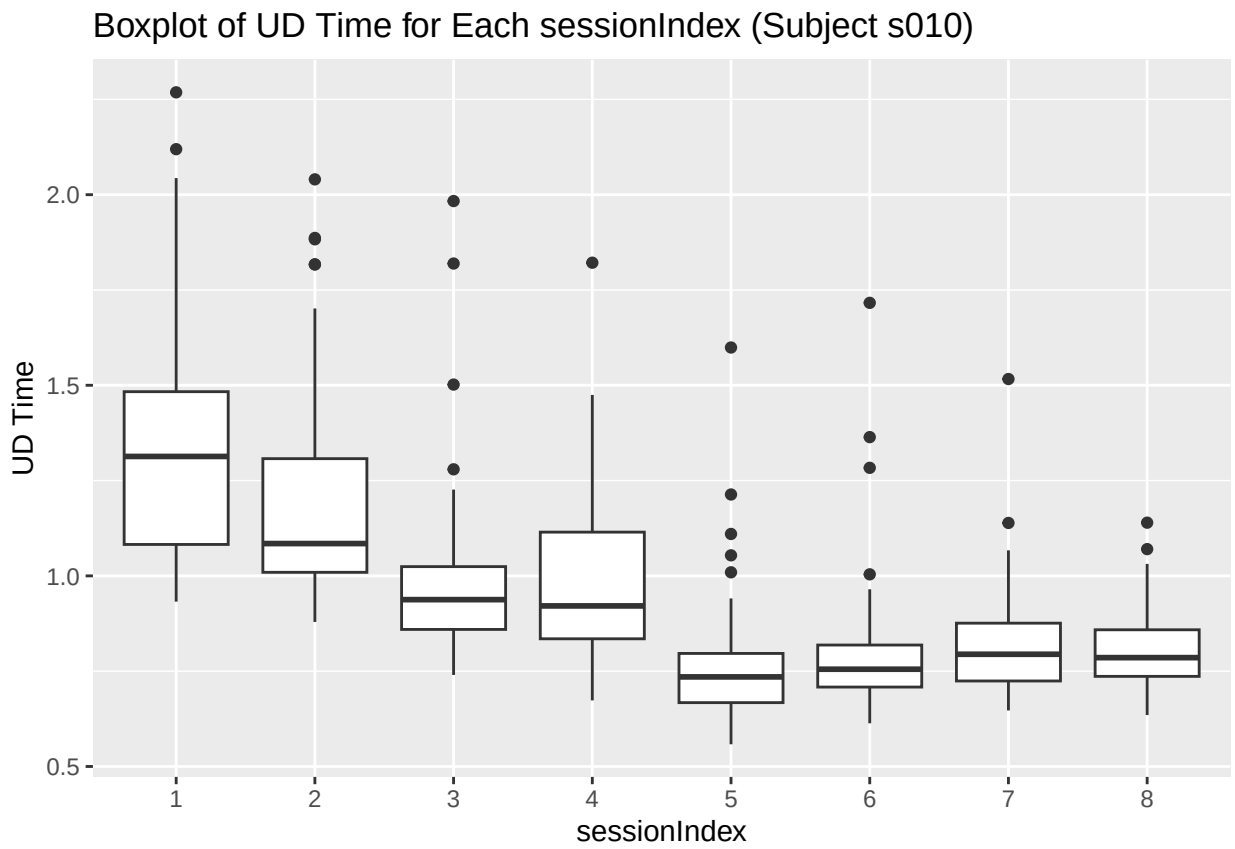

```r
# Subset data for subject 's010'
subset_data_s010 <- data[data$subject == "s010", ]

# Create a box plot for 'total_time' for each 'sessionIndex'
ggplot(subset_data_s010, aes(x = factor(sessionIndex), y = total_time)) +
  geom_boxplot() +
  labs(title = "Boxplot of Total Time for Each sessionIndex (Subject s010)",
       x = "sessionIndex", y = "Total Time")
```



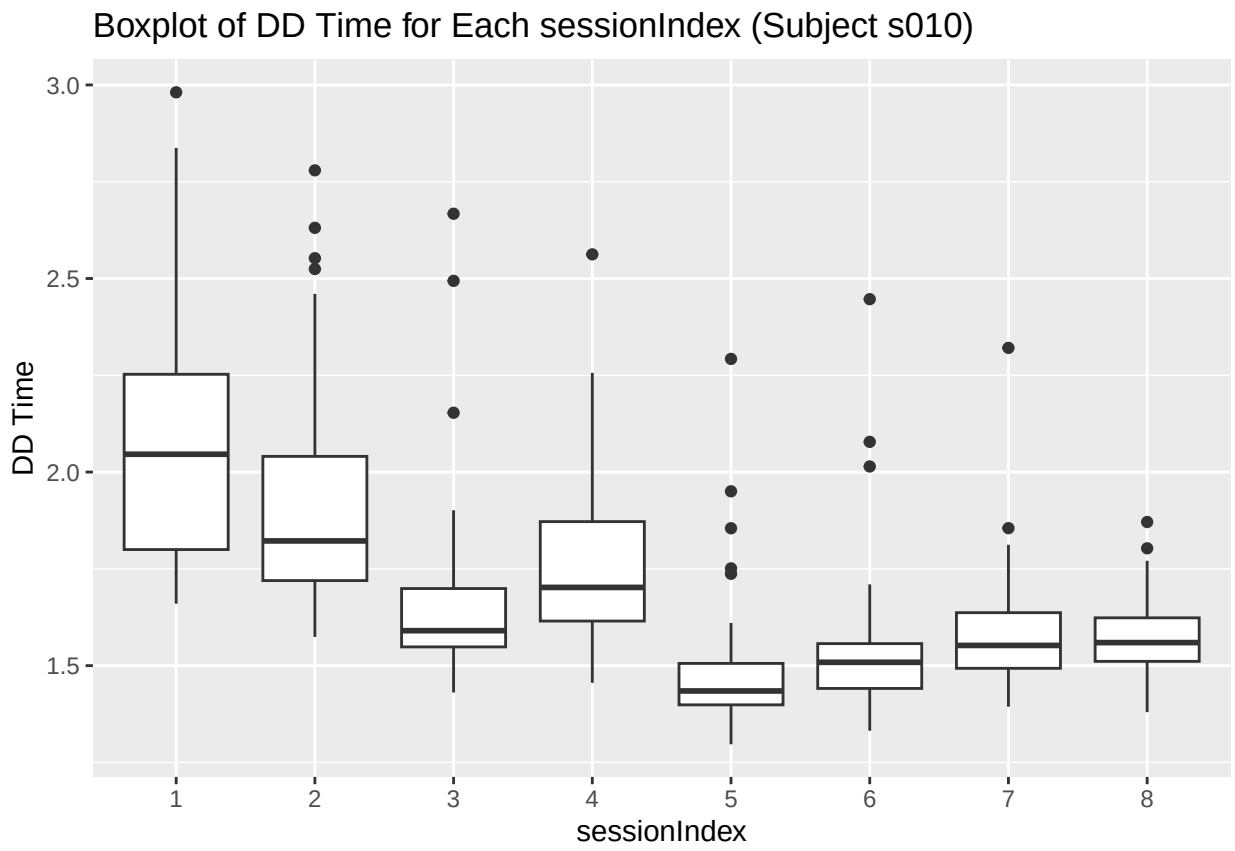

```
# Create a box plot for 'hold_time' for each 'sessionIndex'
ggplot(subset_data_s010, aes(x = factor(sessionIndex), y = hold_time)) +
  geom_boxplot() +
  labs(title = "Boxplot of Hold Time for Each sessionIndex (Subject s010)",
       x = "sessionIndex", y = "Hold Time")
```




```r
# Create a box plot for 'ud_time' for each 'sessionIndex'
ggplot(subset_data_s010, aes(x = factor(sessionIndex), y = ud_time)) +
  geom_boxplot() +
  labs(title = "Boxplot of UD Time for Each sessionIndex (Subject s010)",
       x = "sessionIndex", y = "UD Time")
```




```r
# Create a box plot for 'dd_time' for each 'sessionIndex'
ggplot(subset_data_s010, aes(x = factor(sessionIndex), y = dd_time)) +
  geom_boxplot() +
  labs(title = "Boxplot of DD Time for Each sessionIndex (Subject s010)",
       x = "sessionIndex", y = "DD Time")
```



Sub Task 2

Linear Mixed Effect Random (LMER) Models

```
data$sessionIndex <- as.numeric(data$sessionIndex)

# Function to print model summary
print_model_summary <- function(model, model_name) {
  cat(paste("\nSummary for", model_name, "\n"))
  print(summary(model))
}

# Model 1: Random intercept for subject - total_time
lmer_total_time_1 <- lmer(total_time ~ sessionIndex + (1 | subject), data = data, REML = FALSE)
print_model_summary(lmer_total_time_1, "Model 1: Random intercept for subject - total_time")
```

LMER models for Total time

```
##
## Summary for Model 1: Random intercept for subject - total_time
## Linear mixed model fit by maximum likelihood . t-tests use Satterthwaite's
##   method [lmerModLmerTest]
```

```
## Formula: total_time ~ sessionIndex + (1 | subject)
##    Data: data
##
##      AIC      BIC   logLik deviance df.resid
##  71505.2  71536.9 -35748.6  71497.2    20396
##
## Scaled residuals:
##    Min     1Q Median     3Q    Max
## -3.273 -0.466 -0.129  0.271 44.030
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  subject  (Intercept) 2.967    1.723
##  Residual             1.917    1.385
## Number of obs: 20400, groups:  subject, 51
##
## Fixed effects:
##                Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)   6.200e+00  2.422e-01  5.164e+01   25.61   <2e-16 ***
## sessionIndex -2.507e-01  4.231e-03  2.035e+04  -59.26   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr)
## sessionIndx -0.079
```

```r
# Model 2: Random intercept and random slope for sessionIndex within subject - total_time
lmer_total_time_2 <- lmer(total_time ~ sessionIndex + (sessionIndex | subject), data = data, REML = FALS
print_model_summary(lmer_total_time_2, "Model 2: Random intercept and random slope for sessionIndex with
```

```
##
## Summary for Model 2: Random intercept and random slope for sessionIndex within subject - total_time
## Linear mixed model fit by maximum likelihood . t-tests use Satterthwaite's
##   method [lmerModLmerTest]
## Formula: total_time ~ sessionIndex + (sessionIndex | subject)
##    Data: data
##
##      AIC      BIC   logLik deviance df.resid
##  69021.2  69068.7 -34504.6  69009.2    20394
##
## Scaled residuals:
##    Min     1Q Median     3Q    Max
## -5.256 -0.464 -0.147  0.253 43.783
##
## Random effects:
##  Groups   Name         Variance Std.Dev. Corr
##  subject  (Intercept)  6.26209  2.5024
##           sessionIndex 0.04458  0.2111   -0.88
##  Residual              1.68251  1.2971
## Number of obs: 20400, groups:  subject, 51
##
## Fixed effects:
##              Estimate Std. Error       df t value Pr(>|t|)
```

```
## (Intercept)   6.20032    0.35098 50.98823  17.666  < 2e-16 ***
## sessionIndex -0.25070    0.02983 50.99153  -8.405 3.41e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr)
## sessionIndx -0.880
## optimizer (nloptwrap) convergence code: 0 (OK)
## Model failed to converge with max|grad| = 0.00215959 (tol = 0.002, component 1)
```

```
#Performing likelihood ratio test (LRT) using ANOVA to compare two linear mixed-effects models
anova(lmer_total_time_1, lmer_total_time_2)
```

Comparing the Random Intercept and Random Slope & Intercept models using ANOVA test.

```
## Data: data
## Models:
## lmer_total_time_1: total_time ~ sessionIndex + (1 | subject)
## lmer_total_time_2: total_time ~ sessionIndex + (sessionIndex | subject)
##                   npar   AIC   BIC logLik deviance  Chisq Df Pr(>Chisq)
## lmer_total_time_1    4 71505 71537 -35749    71497
## lmer_total_time_2    6 69021 69069 -34505    69009 2487.9  2  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Model 3: Random intercept for subject - hold_time
lmer_hold_time_1 <- lmer(hold_time ~ sessionIndex + (1 | subject), data = data, REML = FALSE)
print_model_summary(lmer_hold_time_1, "Model 3: Random intercept for subject - hold_time")
```

LMER models for Total hold time

```
##
## Summary for Model 3: Random intercept for subject - hold_time
## Linear mixed model fit by maximum likelihood . t-tests use Satterthwaite's
##   method [lmerModLmerTest]
## Formula: hold_time ~ sessionIndex + (1 | subject)
##    Data: data
##
##      AIC      BIC   logLik deviance df.resid
## -37841.6 -37809.9  18924.8 -37849.6    20396
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -4.5927 -0.5784 -0.0165  0.5396 23.0490
##
## Random effects:
```

```
##  Groups   Name        Variance Std.Dev.
##  subject  (Intercept) 0.043774 0.20922
##  Residual             0.008985 0.09479
## Number of obs: 20400, groups:  subject, 51
##
## Fixed effects:
##               Estimate Std. Error        df t value Pr(>|t|)
## (Intercept)  9.509e-01  2.933e-02 5.120e+01   32.42   <2e-16 ***
## sessionIndex 8.922e-03  2.896e-04 2.035e+04   30.80   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr)
## sessionIndx -0.044
```

```r
# Model 4: Random intercept and random slope for sessionIndex within subject - hold_time
lmer_hold_time_2 <- lmer(hold_time ~ sessionIndex + (sessionIndex | subject), data = data, REML = FALSE)
print_model_summary(lmer_hold_time_2, "Model 4: Random intercept and random slope for sessionIndex with
```

```
##
## Summary for Model 4: Random intercept and random slope for sessionIndex within subject - hold_time
## Linear mixed model fit by maximum likelihood . t-tests use Satterthwaite's
##   method [lmerModLmerTest]
## Formula: hold_time ~ sessionIndex + (sessionIndex | subject)
##    Data: data
##
##      AIC      BIC   logLik deviance df.resid
## -40582.0 -40534.4  20297.0 -40594.0    20394
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -4.8350 -0.5851 -0.0189  0.5440 24.4336
##
## Random effects:
##  Groups   Name         Variance  Std.Dev. Corr
##  subject  (Intercept)  0.0407425 0.20185
##           sessionIndex 0.0002306 0.01518  -0.06
##  Residual              0.0077717 0.08816
## Number of obs: 20400, groups:  subject, 51
##
## Fixed effects:
##               Estimate Std. Error        df t value Pr(>|t|)
## (Intercept)   0.950909   0.028297 50.997979  33.605  < 2e-16 ***
## sessionIndex  0.008922   0.002143 50.994747   4.163 0.000121 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr)
## sessionIndx -0.064
```

```r
#Performing likelihood ratio test (LRT) using ANOVA to compare two linear mixed-effects models
anova(lmer_hold_time_1, lmer_hold_time_2)
```

Comparing the Random Intercept and Random Slope & Intercept models using ANOVA test.

```
## Data: data
## Models:
## lmer_hold_time_1: hold_time ~ sessionIndex + (1 | subject)
## lmer_hold_time_2: hold_time ~ sessionIndex + (sessionIndex | subject)
##                   npar    AIC    BIC logLik deviance  Chisq Df Pr(>Chisq)
## lmer_hold_time_1    4 -37842 -37810  18925   -37850
## lmer_hold_time_2    6 -40582 -40534  20297   -40594 2744.4  2  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# Model 5: Random intercept for subject - ud_time
lmer_ud_time_1 <- lmer(ud_time ~ sessionIndex + (1 | subject), data = data, REML = FALSE)
print_model_summary(lmer_ud_time_1, "Model 5: Random intercept for subject - ud_time")
```

LMER models for Total key up and key down time

```
##
## Summary for Model 5: Random intercept for subject - ud_time
## Linear mixed model fit by maximum likelihood . t-tests use Satterthwaite's
##   method [lmerModLmerTest]
## Formula: ud_time ~ sessionIndex + (1 | subject)
##    Data: data
##
##      AIC      BIC   logLik deviance df.resid
##  43115.5  43147.2 -21553.8  43107.5    20396
##
## Scaled residuals:
##    Min     1Q Median     3Q    Max
## -3.206 -0.468 -0.130  0.272 44.166
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  subject  (Intercept) 0.8137   0.9021
##  Residual             0.4766   0.6904
## Number of obs: 20400, groups:  subject, 51
##
## Fixed effects:
##                Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)   2.193e+00  1.268e-01  5.158e+01   17.30   <2e-16 ***
## sessionIndex -1.341e-01  2.110e-03  2.035e+04  -63.59   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Correlation of Fixed Effects:
##             (Intr)
## sessionIndx -0.075
```

```
# Model 6: Random intercept and random slope for sessionIndex within subject - ud_time
lmer_ud_time_2 <- lmer(ud_time ~ sessionIndex + (sessionIndex | subject), data = data, REML = FALSE)
```

```
## Warning in checkConv(attr(opt, "derivs"), opt$par, ctrl = control$checkConv, :
## Model failed to converge with max|grad| = 0.00301658 (tol = 0.002, component 1)
```

```
print_model_summary(lmer_ud_time_2, "Model 6: Random intercept and random slope for sessionIndex within
```

```
##
## Summary for Model 6: Random intercept and random slope for sessionIndex within subject - ud_time
## Linear mixed model fit by maximum likelihood . t-tests use Satterthwaite's
##   method [lmerModLmerTest]
## Formula: ud_time ~ sessionIndex + (sessionIndex | subject)
##    Data: data
##
##      AIC      BIC   logLik deviance df.resid
##  40735.6  40783.1 -20361.8  40723.6    20394
##
## Scaled residuals:
##    Min     1Q Median     3Q    Max
## -5.165 -0.457 -0.145  0.255 43.919
##
## Random effects:
##  Groups   Name         Variance Std.Dev. Corr
##  subject  (Intercept)  1.62321  1.2741
##           sessionIndex 0.01068  0.1033   -0.87
##  Residual              0.42039  0.6484
## Number of obs: 20400, groups:  subject, 51
##
## Fixed effects:
##              Estimate Std. Error       df t value Pr(>|t|)
## (Intercept)   2.19282    0.17868 51.00102  12.272  < 2e-16 ***
## sessionIndex -0.13414    0.01461 50.99931  -9.185 2.16e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr)
## sessionIndx -0.863
## optimizer (nloptwrap) convergence code: 0 (OK)
## Model failed to converge with max|grad| = 0.00301658 (tol = 0.002, component 1)
```

```
#Performing likelihood ratio test (LRT) using ANOVA to compare two linear mixed-effects models
anova(lmer_ud_time_1, lmer_ud_time_2)
```

Comparing the Random Intercept and Random Slope & Intercept models using ANOVA test.

```
## Data: data
## Models:
## lmer_ud_time_1: ud_time ~ sessionIndex + (1 | subject)
## lmer_ud_time_2: ud_time ~ sessionIndex + (sessionIndex | subject)
##                npar   AIC   BIC logLik deviance  Chisq Df Pr(>Chisq)
## lmer_ud_time_1    4 43116 43147 -21554    43108
## lmer_ud_time_2    6 40736 40783 -20362    40724 2383.9  2  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# Model 7: Random intercept for subject - dd_time
lmer_dd_time_1 <- lmer(dd_time ~ sessionIndex + (1 | subject), data = data, REML = FALSE)
print_model_summary(lmer_dd_time_1, "Model 7: Random intercept for subject - dd_time")
```

LMER models for Total key down and key down time

```
##
## Summary for Model 7: Random intercept for subject - dd_time
## Linear mixed model fit by maximum likelihood . t-tests use Satterthwaite's
##   method [lmerModLmerTest]
## Formula: dd_time ~ sessionIndex + (1 | subject)
##    Data: data
##
##      AIC      BIC   logLik deviance df.resid
##  43176.4  43208.1 -21584.2  43168.4    20396
##
## Scaled residuals:
##    Min     1Q Median     3Q    Max
## -3.294 -0.464 -0.129  0.269 44.079
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  subject  (Intercept) 0.7471   0.8643
##  Residual             0.4781   0.6915
## Number of obs: 20400, groups:  subject, 51
##
## Fixed effects:
##                Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)   3.057e+00  1.215e-01  5.163e+01   25.16   <2e-16 ***
## sessionIndex -1.255e-01  2.113e-03  2.035e+04  -59.39   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr)
## sessionIndx -0.078
```

```r
# Model 8: Random intercept and random slope for sessionIndex within subject - dd_time
lmer_dd_time_2 <- lmer(dd_time ~ sessionIndex + (sessionIndex | subject), data = data, REML = FALSE)
print_model_summary(lmer_dd_time_2, "Model 8: Random intercept and random slope for sessionIndex within
```

```
##
## Summary for Model 8: Random intercept and random slope for sessionIndex within subject - dd_time
## Linear mixed model fit by maximum likelihood . t-tests use Satterthwaite's
##   method [lmerModLmerTest]
## Formula: dd_time ~ sessionIndex + (sessionIndex | subject)
##    Data: data
##
##      AIC      BIC   logLik deviance df.resid
##  40697.4  40745.0 -20342.7  40685.4    20394
##
## Scaled residuals:
##    Min     1Q Median     3Q    Max
## -5.265 -0.461 -0.148  0.252 43.829
##
## Random effects:
##  Groups   Name         Variance Std.Dev. Corr
##  subject  (Intercept)  1.5708   1.2533
##           sessionIndex 0.0111   0.1053   -0.88
##  Residual              0.4197   0.6479
## Number of obs: 20400, groups:  subject, 51
##
## Fixed effects:
##              Estimate Std. Error       df t value Pr(>|t|)
## (Intercept)   3.05660    0.17578 51.00281  17.388  < 2e-16 ***
## sessionIndex -0.12548    0.01488 51.00026  -8.432 3.09e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr)
## sessionIndx -0.880
## optimizer (nloptwrap) convergence code: 0 (OK)
## Model failed to converge with max|grad| = 0.00450988 (tol = 0.002, component 1)
```

```
#Performing likelihood ratio test (LRT) using ANOVA to compare two linear mixed-effects models
anova(lmer_dd_time_1, lmer_dd_time_2)
```

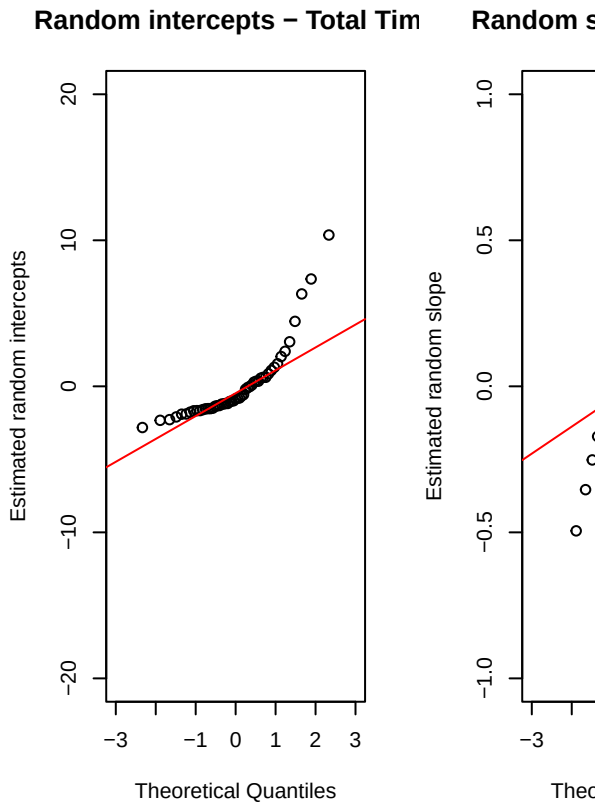
Comparing the Random Intercept and Random Slope & Intercept models using ANOVA test.

```
## Data: data
## Models:
## lmer_dd_time_1: dd_time ~ sessionIndex + (1 | subject)
## lmer_dd_time_2: dd_time ~ sessionIndex + (sessionIndex | subject)
##                 npar   AIC   BIC logLik deviance Chisq Df Pr(>Chisq)
## lmer_dd_time_1    4 43176 43208 -21584    43168
## lmer_dd_time_2    6 40697 40745 -20343    40685  2483  2  < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Sub Task 3

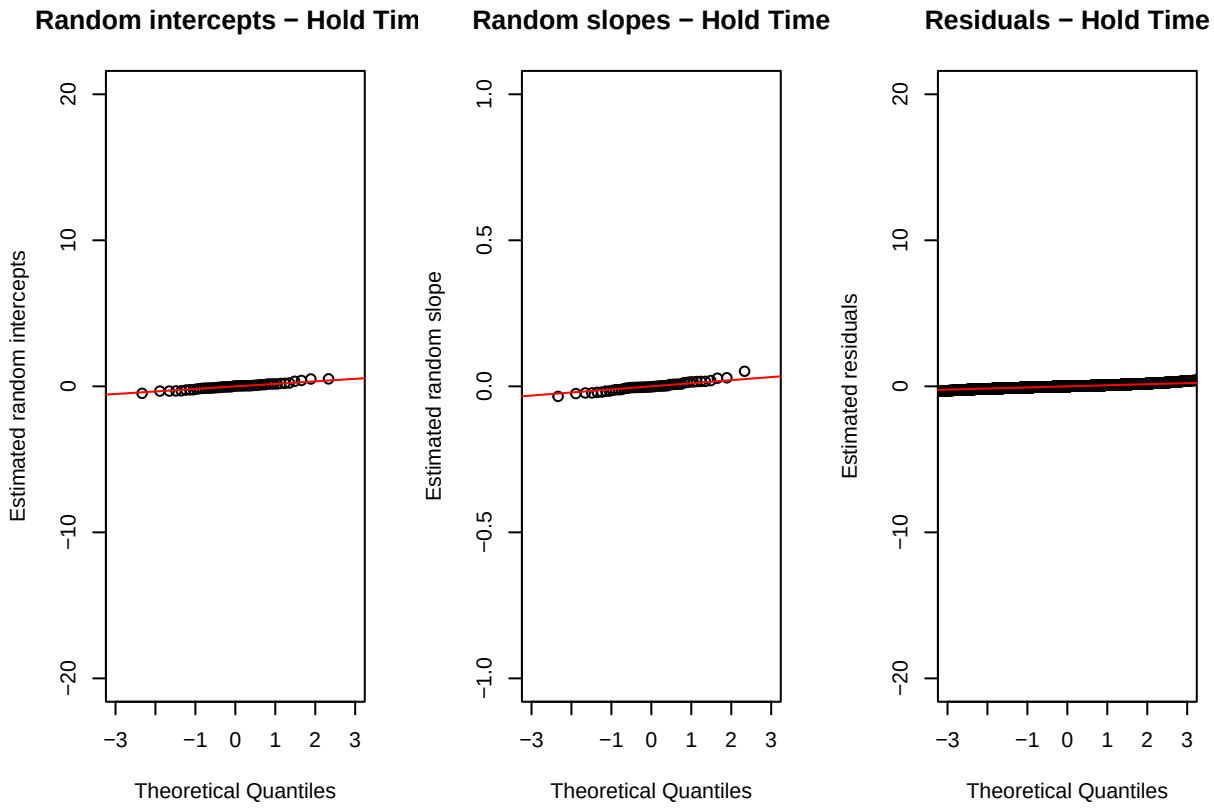
Post Hoc Analysis

```r
# Function to create qq plot matrix
qq_plot_matrix <- function(lmer_model, title, ylim) {
  # Extract random effects and residuals
  qint <- ranef(lmer_model)$subject[["(Intercept)"]]

  # Check if the model includes a random slope
  if ("sessionIndex" %in% names(ranef(lmer_model)$subject)) {
    qslo <- ranef(lmer_model)$subject[["sessionIndex"]]
  } else {
    # If no random slope, create an empty vector
    qslo <- numeric(length(qint))
  }

  qres <- resid(lmer_model)

  # Create qq plot matrix
  par(mfrow = c(1, 3))

  # QQ Plot for Random Intercepts
  qqnorm(qint, ylab = "Estimated random intercepts",
         xlim = c(-3, 3), ylim = ylim,
         main = paste("Random intercepts -", title))
  qqline(qint, col = "red", lvd = 3)

  # Check if qslo is not empty
  if (length(qslo) > 0) {
    # QQ Plot for Random Slopes
    qqnorm(qslo, ylab = "Estimated random slope",
           xlim = c(-3, 3), ylim = c(-1, 1),
           main = paste("Random slopes -", title))
    qqline(qslo, col = "red", lvd = 3)
  } else {
    cat("No random slope for sessionIndex in the model.\n")
  }

  # QQ Plot for Residuals
  qqnorm(qres, ylab = "Estimated residuals",
         xlim = c(-3, 3), ylim = ylim,
         main = paste("Residuals -", title))
  qqline(qres, col = "red", lvd = 3)
}

# Create QQ Plot Matrix for total_time
qq_plot_matrix(lmer_total_time_2, "Total Time", c(-20, 20))
```

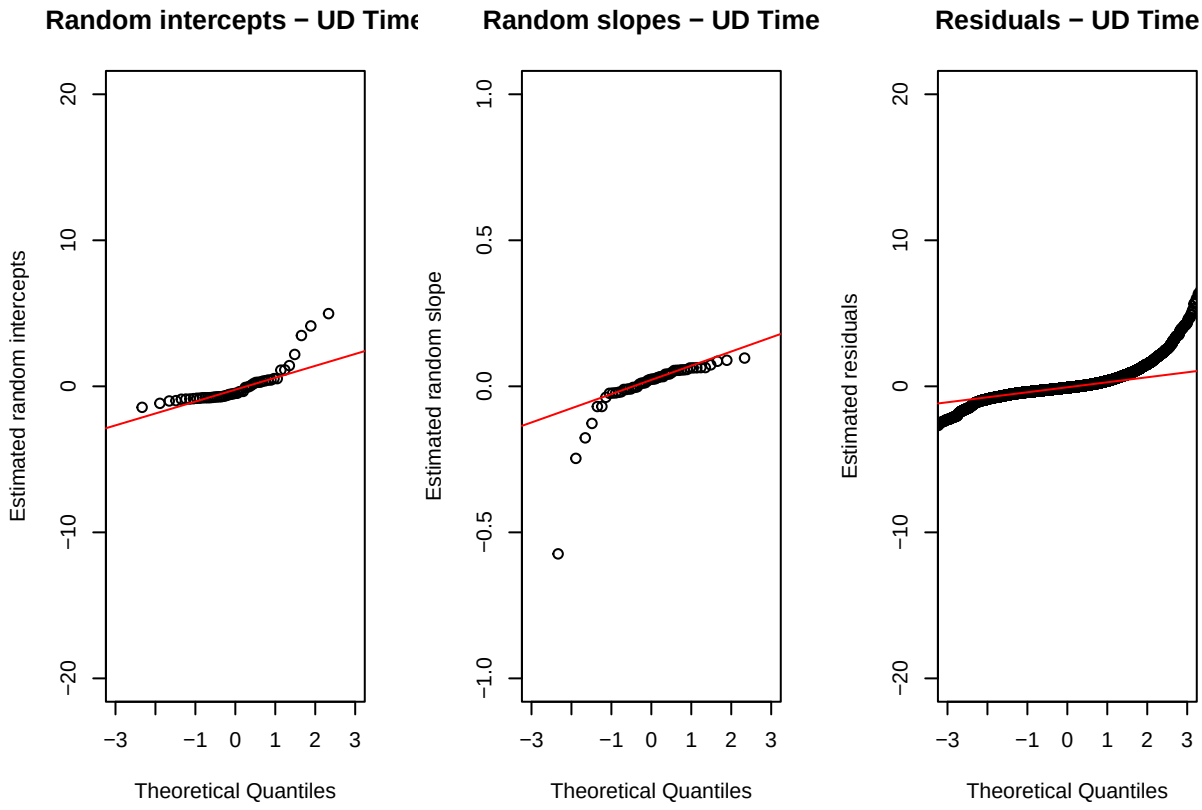


QQ Plot to verify the normality of random effects and residuals

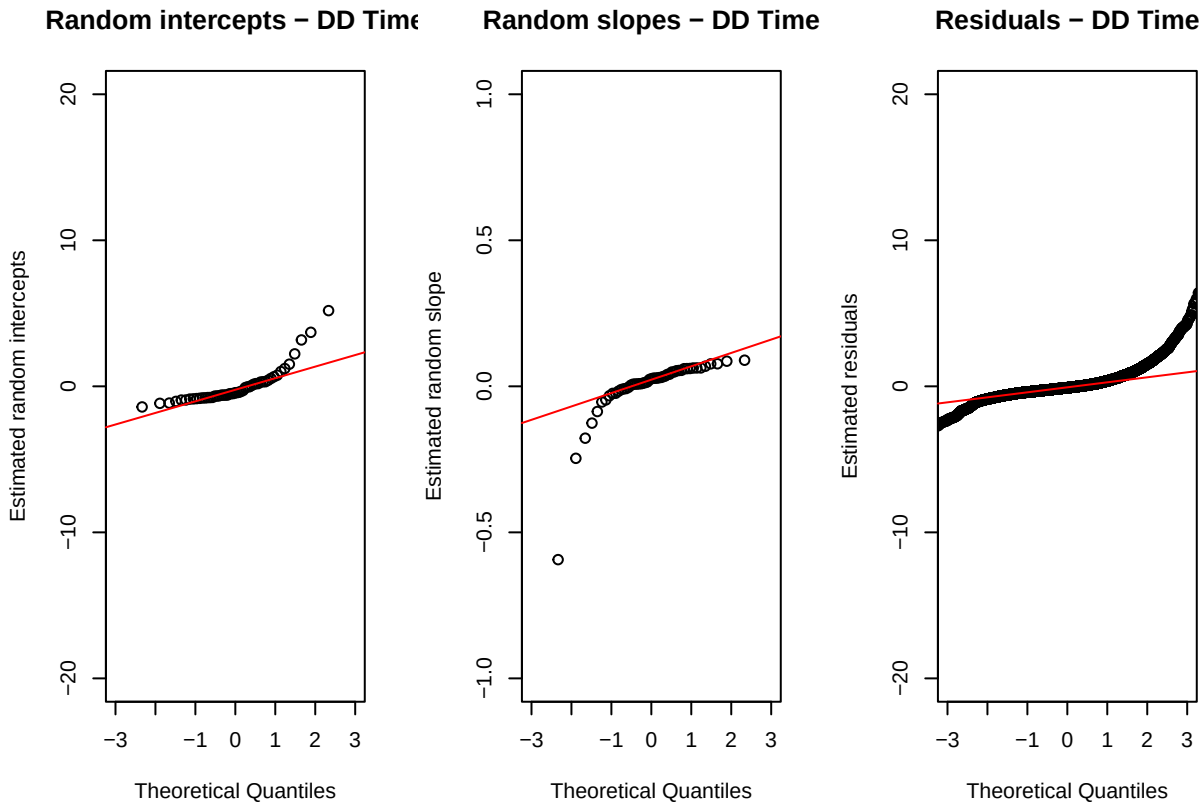
```
# Create QQ Plot Matrix for hold_time
qq_plot_matrix(lmer_hold_time_2, "Hold Time", c(-20, 20))
```




```r
# Create QQ Plot Matrix for ud_time
qq_plot_matrix(lmer_ud_time_2, "UD Time", c(-20, 20))
```




```r
# Create QQ Plot Matrix for dd_time
qq_plot_matrix(lmer_dd_time_2, "DD Time", c(-20, 20))
```




```r
# Function to perform Shapiro-Wilk test for normality on residuals and random effects
shapiro_test <- function(lmer_model, title, sample_size = 5000) {
  cat("Shapiro-Wilk Test for Normality -", title, "\n")

  # Extract random effects and residuals
  qint <- ranef(lmer_model)$subject[["(Intercept)"]]
  qslo <- ranef(lmer_model)$subject[["sessionIndex"]]
  qres <- resid(lmer_model)

  # Perform Shapiro-Wilk test for random intercepts
  cat("\nRandom Intercepts:\n")
  shapiro_result_int <- shapiro.test(qint)
  print(shapiro_result_int)
  if (shapiro_result_int$p.value < 0.05) {
    cat("The random intercepts are not normally distributed (p-value < 0.05)\n")
  } else {
    cat("The random intercepts are normally distributed (p-value >= 0.05)\n")
  }

  # Perform Shapiro-Wilk test for random slopes
  cat("\nRandom Slopes:\n")
  shapiro_result_slo <- shapiro.test(qslo)
  print(shapiro_result_slo)
```

```
  if (shapiro_result_slo$p.value < 0.05) {
    cat("The random slopes are not normally distributed (p-value < 0.05)\n")
  } else {
    cat("The random slopes are normally distributed (p-value >= 0.05)\n")
  }

  # Perform Shapiro-Wilk test for residuals
  cat("\nResiduals:\n")
  if (length(qres) > sample_size) {
    qres_sample <- sample(qres, sample_size)
  } else {
    qres_sample <- qres
  }
  shapiro_result_res <- shapiro.test(qres_sample)
  print(shapiro_result_res)
  if (shapiro_result_res$p.value < 0.05) {
    cat("The residuals are not normally distributed (p-value < 0.05)\n")
  } else {
    cat("The residuals are normally distributed (p-value >= 0.05)\n")
  }

  cat("\n")
}
```

Shapiro-Wilk test to test the normality on residuals and random effects

Results of Shapiro Wilk Test

```
# Perform Shapiro-Wilk test for each model and each variable
shapiro_test(lmer_total_time_2, "Total Time")
```

For Total time (Random Slope & Intercept Model)

```
## Shapiro-Wilk Test for Normality - Total Time
##
## Random Intercepts:
##
##  Shapiro-Wilk normality test
##
## data:  qint
## W = 0.75528, p-value = 7.616e-08
##
## The random intercepts are not normally distributed (p-value < 0.05)
##
## Random Slopes:
##
##  Shapiro-Wilk normality test
##
## data:  qslo
```

```
## W = 0.59092, p-value = 1.1e-10
##
## The random slopes are not normally distributed (p-value < 0.05)
##
## Residuals:
##
##  Shapiro-Wilk normality test
##
## data:  qres_sample
## W = 0.79773, p-value < 2.2e-16
##
## The residuals are not normally distributed (p-value < 0.05)
```

```
shapiro_test(lmer_hold_time_2, "Hold Time")
```

For Total hold time (Random Slope & Intercept Model)

```
## Shapiro-Wilk Test for Normality - Hold Time
##
## Random Intercepts:
##
##  Shapiro-Wilk normality test
##
## data:  qint
## W = 0.97661, p-value = 0.4063
##
## The random intercepts are normally distributed (p-value >= 0.05)
##
## Random Slopes:
##
##  Shapiro-Wilk normality test
##
## data:  qslo
## W = 0.96422, p-value = 0.1262
##
## The random slopes are normally distributed (p-value >= 0.05)
##
## Residuals:
##
##  Shapiro-Wilk normality test
##
## data:  qres_sample
## W = 0.97969, p-value < 2.2e-16
##
## The residuals are not normally distributed (p-value < 0.05)
```

```
shapiro_test(lmer_ud_time_2, "UD Time")
```

For Total Key up and Key down time (Random Slope & Intercept Model)

```
## Shapiro-Wilk Test for Normality - UD Time
##
## Random Intercepts:
##
##  Shapiro-Wilk normality test
##
## data:  qint
## W = 0.73207, p-value = 2.617e-08
##
## The random intercepts are not normally distributed (p-value < 0.05)
##
## Random Slopes:
##
##  Shapiro-Wilk normality test
##
## data:  qslo
## W = 0.61024, p-value = 2.141e-10
##
## The random slopes are not normally distributed (p-value < 0.05)
##
## Residuals:
##
##  Shapiro-Wilk normality test
##
## data:  qres_sample
## W = 0.77579, p-value < 2.2e-16
##
## The residuals are not normally distributed (p-value < 0.05)
```

```
shapiro_test(lmer_dd_time_2, "DD Time")
```

For Total key down down time (Random Slope & Intercept Model)

```
## Shapiro-Wilk Test for Normality - DD Time
##
## Random Intercepts:
##
##  Shapiro-Wilk normality test
##
## data:  qint
## W = 0.75405, p-value = 7.187e-08
##
## The random intercepts are not normally distributed (p-value < 0.05)
##
## Random Slopes:
##
##  Shapiro-Wilk normality test
##
## data:  qslo
```

```
## W = 0.58844, p-value = 1.011e-10
##
## The random slopes are not normally distributed (p-value < 0.05)
##
## Residuals:
##
##  Shapiro-Wilk normality test
##
## data:  qres_sample
## W = 0.81504, p-value < 2.2e-16
##
## The residuals are not normally distributed (p-value < 0.05)
```

Pairwise Comparsion

```
data$sessionIndex <- as.factor(data$sessionIndex)

# Model 2: Random intercept and random slope for sessionIndex within subject - total_time_2
lmer_total_time_2 <- lmer(total_time ~ sessionIndex + (sessionIndex | subject), data = data, REML = FALS

# Model 2: Random intercept and random slope for sessionIndex within subject - hold_time_2
lmer_hold_time_2 <- lmer(hold_time ~ sessionIndex + (sessionIndex | subject), data = data, REML = FALSE)

# Model 2: Random intercept and random slope for sessionIndex within subject - ud_time_2
lmer_ud_time_2 <- lmer(ud_time ~ sessionIndex + (sessionIndex | subject), data = data, REML = FALSE)

# Model 2: Random intercept and random slope for sessionIndex within subject - dd_time_2
lmer_dd_time_2 <- lmer(dd_time ~ sessionIndex + (sessionIndex | subject), data = data, REML = FALSE)
```

Converting sessionIndex to factor in order to compare the marginal means.

Comparing the marginal means of summarized times for each sessionIndex.

```
# Perform pairwise comparisons for total_time_2
means_lmer_total_time_2 <- emmeans(lmer_total_time_2, specs = ~sessionIndex)
pairwise_comparisons_total_time_2 <- pairs(means_lmer_total_time_2)
cat("\nPairwise Comparisons for Total Time:\n")
```

For Total time

```
##
## Pairwise Comparisons for Total Time:
```

```
print(pairwise_comparisons_total_time_2)
```

```
##  contrast                      estimate     SE  df z.ratio p.value
##  sessionIndex1 - sessionIndex2  0.88941 0.1950 Inf   4.570  0.0001
##  sessionIndex1 - sessionIndex3  1.13513 0.2030 Inf   5.579  <.0001
##  sessionIndex1 - sessionIndex4  1.46329 0.2290 Inf   6.396  <.0001
##  sessionIndex1 - sessionIndex5  1.64256 0.2540 Inf   6.476  <.0001
##  sessionIndex1 - sessionIndex6  1.81148 0.2650 Inf   6.824  <.0001
##  sessionIndex1 - sessionIndex7  1.93874 0.2670 Inf   7.258  <.0001
##  sessionIndex1 - sessionIndex8  1.94345 0.2620 Inf   7.412  <.0001
##  sessionIndex2 - sessionIndex3  0.24572 0.0763 Inf   3.223  0.0278
##  sessionIndex2 - sessionIndex4  0.57388 0.0777 Inf   7.382  <.0001
##  sessionIndex2 - sessionIndex5  0.75315 0.0875 Inf   8.612  <.0001
##  sessionIndex2 - sessionIndex6  0.92207 0.0947 Inf   9.740  <.0001
##  sessionIndex2 - sessionIndex7  1.04933 0.1160 Inf   9.038  <.0001
##  sessionIndex2 - sessionIndex8  1.05404 0.1180 Inf   8.968  <.0001
##  sessionIndex3 - sessionIndex4  0.32816 0.0765 Inf   4.287  0.0005
##  sessionIndex3 - sessionIndex5  0.50742 0.0942 Inf   5.388  <.0001
##  sessionIndex3 - sessionIndex6  0.67635 0.0952 Inf   7.106  <.0001
##  sessionIndex3 - sessionIndex7  0.80361 0.1140 Inf   7.020  <.0001
##  sessionIndex3 - sessionIndex8  0.80832 0.1050 Inf   7.712  <.0001
##  sessionIndex4 - sessionIndex5  0.17927 0.0574 Inf   3.122  0.0381
##  sessionIndex4 - sessionIndex6  0.34819 0.0734 Inf   4.743  0.0001
##  sessionIndex4 - sessionIndex7  0.47545 0.1040 Inf   4.567  0.0001
##  sessionIndex4 - sessionIndex8  0.48017 0.0921 Inf   5.216  <.0001
##  sessionIndex5 - sessionIndex6  0.16892 0.0538 Inf   3.143  0.0357
##  sessionIndex5 - sessionIndex7  0.29618 0.0880 Inf   3.365  0.0175
##  sessionIndex5 - sessionIndex8  0.30090 0.0872 Inf   3.451  0.0130
##  sessionIndex6 - sessionIndex7  0.12726 0.0667 Inf   1.909  0.5448
##  sessionIndex6 - sessionIndex8  0.13197 0.0848 Inf   1.556  0.7768
##  sessionIndex7 - sessionIndex8  0.00472 0.0699 Inf   0.067  1.0000
##
## Degrees-of-freedom method: asymptotic
## P value adjustment: tukey method for comparing a family of 8 estimates
```

```
# Perform pairwise comparisons for hold_time_2
means_lmer_hold_time_2 <- emmeans(lmer_hold_time_2, specs = ~sessionIndex)
pairwise_comparisons_hold_time_2 <- pairs(means_lmer_hold_time_2)
cat("\nPairwise Comparisons for Hold Time:\n")
```

For Total hold time

```
##
## Pairwise Comparisons for Hold Time:
```

```
print(pairwise_comparisons_hold_time_2)
```

```
##  contrast                       estimate      SE  df z.ratio p.value
##  sessionIndex1 - sessionIndex2 -0.031546 0.01150 Inf  -2.751  0.1077
##  sessionIndex1 - sessionIndex3 -0.025306 0.01140 Inf  -2.216  0.3417
##  sessionIndex1 - sessionIndex4 -0.028228 0.01140 Inf  -2.479  0.2044
##  sessionIndex1 - sessionIndex5 -0.031036 0.01330 Inf  -2.331  0.2764
```

```
##  sessionIndex1 - sessionIndex6 -0.060168 0.01440 Inf  -4.182  0.0008
##  sessionIndex1 - sessionIndex7 -0.060898 0.01440 Inf  -4.231  0.0006
##  sessionIndex1 - sessionIndex8 -0.070757 0.01690 Inf  -4.181  0.0008
##  sessionIndex2 - sessionIndex3  0.006240 0.01110 Inf   0.561  0.9993
##  sessionIndex2 - sessionIndex4  0.003318 0.01010 Inf   0.329  1.0000
##  sessionIndex2 - sessionIndex5  0.000511 0.01240 Inf   0.041  1.0000
##  sessionIndex2 - sessionIndex6 -0.028621 0.01250 Inf  -2.285  0.3018
##  sessionIndex2 - sessionIndex7 -0.029352 0.01530 Inf  -1.919  0.5379
##  sessionIndex2 - sessionIndex8 -0.039211 0.01680 Inf  -2.330  0.2773
##  sessionIndex3 - sessionIndex4 -0.002922 0.00918 Inf  -0.318  1.0000
##  sessionIndex3 - sessionIndex5 -0.005730 0.01110 Inf  -0.516  0.9996
##  sessionIndex3 - sessionIndex6 -0.034861 0.01140 Inf  -3.045  0.0480
##  sessionIndex3 - sessionIndex7 -0.035592 0.01440 Inf  -2.468  0.2094
##  sessionIndex3 - sessionIndex8 -0.045451 0.01610 Inf  -2.818  0.0906
##  sessionIndex4 - sessionIndex5 -0.002808 0.00982 Inf  -0.286  1.0000
##  sessionIndex4 - sessionIndex6 -0.031940 0.01140 Inf  -2.812  0.0920
##  sessionIndex4 - sessionIndex7 -0.032670 0.01240 Inf  -2.636  0.1430
##  sessionIndex4 - sessionIndex8 -0.042529 0.01470 Inf  -2.886  0.0753
##  sessionIndex5 - sessionIndex6 -0.029132 0.00977 Inf  -2.983  0.0574
##  sessionIndex5 - sessionIndex7 -0.029862 0.01200 Inf  -2.493  0.1982
##  sessionIndex5 - sessionIndex8 -0.039721 0.01610 Inf  -2.470  0.2085
##  sessionIndex6 - sessionIndex7 -0.000730 0.01210 Inf  -0.060  1.0000
##  sessionIndex6 - sessionIndex8 -0.010589 0.01540 Inf  -0.686  0.9974
##  sessionIndex7 - sessionIndex8 -0.009859 0.01240 Inf  -0.796  0.9934
##
## Degrees-of-freedom method: asymptotic
## P value adjustment: tukey method for comparing a family of 8 estimates
```

```
# Perform pairwise comparisons for ud_time_2
means_lmer_ud_time_2 <- emmeans(lmer_ud_time_2, specs = ~sessionIndex)
pairwise_comparisons_ud_time_2 <- pairs(means_lmer_ud_time_2)
cat("\nPairwise Comparisons for UD Time:\n")
```

For Total key up and key down time

```
##
## Pairwise Comparisons for UD Time:
```

```
print(pairwise_comparisons_ud_time_2)
```

```
##  contrast                      estimate     SE  df z.ratio p.value
##  sessionIndex1 - sessionIndex2   0.4757 0.0962 Inf   4.943  <.0001
##  sessionIndex1 - sessionIndex3   0.5931 0.1020 Inf   5.840  <.0001
##  sessionIndex1 - sessionIndex4   0.7611 0.1130 Inf   6.721  <.0001
##  sessionIndex1 - sessionIndex5   0.8534 0.1260 Inf   6.765  <.0001
##  sessionIndex1 - sessionIndex6   0.9652 0.1300 Inf   7.443  <.0001
##  sessionIndex1 - sessionIndex7   1.0302 0.1320 Inf   7.826  <.0001
##  sessionIndex1 - sessionIndex8   1.0409 0.1280 Inf   8.119  <.0001
##  sessionIndex2 - sessionIndex3   0.1174 0.0409 Inf   2.874  0.0779
##  sessionIndex2 - sessionIndex4   0.2854 0.0389 Inf   7.333  <.0001
```

```
##  sessionIndex2 - sessionIndex5   0.3777 0.0451 Inf   8.383  <.0001
##  sessionIndex2 - sessionIndex6   0.4896 0.0456 Inf  10.746  <.0001
##  sessionIndex2 - sessionIndex7   0.5546 0.0563 Inf   9.844  <.0001
##  sessionIndex2 - sessionIndex8   0.5653 0.0577 Inf   9.795  <.0001
##  sessionIndex3 - sessionIndex4   0.1680 0.0369 Inf   4.549  0.0001
##  sessionIndex3 - sessionIndex5   0.2603 0.0451 Inf   5.772  <.0001
##  sessionIndex3 - sessionIndex6   0.3722 0.0471 Inf   7.904  <.0001
##  sessionIndex3 - sessionIndex7   0.4372 0.0565 Inf   7.734  <.0001
##  sessionIndex3 - sessionIndex8   0.4478 0.0504 Inf   8.882  <.0001
##  sessionIndex4 - sessionIndex5   0.0923 0.0283 Inf   3.259  0.0248
##  sessionIndex4 - sessionIndex6   0.2042 0.0361 Inf   5.662  <.0001
##  sessionIndex4 - sessionIndex7   0.2691 0.0511 Inf   5.266  <.0001
##  sessionIndex4 - sessionIndex8   0.2798 0.0447 Inf   6.254  <.0001
##  sessionIndex5 - sessionIndex6   0.1119 0.0257 Inf   4.353  0.0004
##  sessionIndex5 - sessionIndex7   0.1769 0.0422 Inf   4.189  0.0007
##  sessionIndex5 - sessionIndex8   0.1875 0.0403 Inf   4.649  0.0001
##  sessionIndex6 - sessionIndex7   0.0650 0.0300 Inf   2.163  0.3746
##  sessionIndex6 - sessionIndex8   0.0757 0.0385 Inf   1.966  0.5050
##  sessionIndex7 - sessionIndex8   0.0107 0.0313 Inf   0.341  1.0000
##
## Degrees-of-freedom method: asymptotic
## P value adjustment: tukey method for comparing a family of 8 estimates
```

```
# Perform pairwise comparisons for dd_time_2
means_lmer_dd_time_2 <- emmeans(lmer_dd_time_2, specs = ~sessionIndex)
pairwise_comparisons_dd_time_2 <- pairs(means_lmer_dd_time_2)
cat("\nPairwise Comparisons for DD Time:\n")
```

For Total key down and key down time

```
##
## Pairwise Comparisons for DD Time:
```

```
print(pairwise_comparisons_dd_time_2)
```

```
##  contrast                      estimate     SE  df z.ratio p.value
##  sessionIndex1 - sessionIndex2   0.4453 0.0974 Inf   4.574  0.0001
##  sessionIndex1 - sessionIndex3   0.5674 0.1020 Inf   5.577  <.0001
##  sessionIndex1 - sessionIndex4   0.7304 0.1140 Inf   6.384  <.0001
##  sessionIndex1 - sessionIndex5   0.8202 0.1270 Inf   6.462  <.0001
##  sessionIndex1 - sessionIndex6   0.9064 0.1330 Inf   6.833  <.0001
##  sessionIndex1 - sessionIndex7   0.9694 0.1340 Inf   7.259  <.0001
##  sessionIndex1 - sessionIndex8   0.9733 0.1310 Inf   7.433  <.0001
##  sessionIndex2 - sessionIndex3   0.1221 0.0382 Inf   3.194  0.0305
##  sessionIndex2 - sessionIndex4   0.2851 0.0389 Inf   7.338  <.0001
##  sessionIndex2 - sessionIndex5   0.3749 0.0436 Inf   8.589  <.0001
##  sessionIndex2 - sessionIndex6   0.4611 0.0471 Inf   9.783  <.0001
##  sessionIndex2 - sessionIndex7   0.5241 0.0578 Inf   9.062  <.0001
##  sessionIndex2 - sessionIndex8   0.5280 0.0584 Inf   9.037  <.0001
##  sessionIndex3 - sessionIndex4   0.1630 0.0382 Inf   4.264  0.0005
```

```
##  sessionIndex3 - sessionIndex5   0.2529 0.0471 Inf   5.369  <.0001
##  sessionIndex3 - sessionIndex6   0.3390 0.0476 Inf   7.121  <.0001
##  sessionIndex3 - sessionIndex7   0.4020 0.0571 Inf   7.044  <.0001
##  sessionIndex3 - sessionIndex8   0.4059 0.0521 Inf   7.787  <.0001
##  sessionIndex4 - sessionIndex5   0.0898 0.0285 Inf   3.152  0.0347
##  sessionIndex4 - sessionIndex6   0.1760 0.0365 Inf   4.817  <.0001
##  sessionIndex4 - sessionIndex7   0.2390 0.0518 Inf   4.610  0.0001
##  sessionIndex4 - sessionIndex8   0.2429 0.0456 Inf   5.323  <.0001
##  sessionIndex5 - sessionIndex6   0.0862 0.0266 Inf   3.241  0.0262
##  sessionIndex5 - sessionIndex7   0.1492 0.0439 Inf   3.399  0.0156
##  sessionIndex5 - sessionIndex8   0.1531 0.0430 Inf   3.556  0.0090
##  sessionIndex6 - sessionIndex7   0.0630 0.0330 Inf   1.907  0.5462
##  sessionIndex6 - sessionIndex8   0.0669 0.0419 Inf   1.595  0.7533
##  sessionIndex7 - sessionIndex8   0.0039 0.0345 Inf   0.113  1.0000
##
## Degrees-of-freedom method: asymptotic
## P value adjustment: tukey method for comparing a family of 8 estimates
```

Sub Task 4

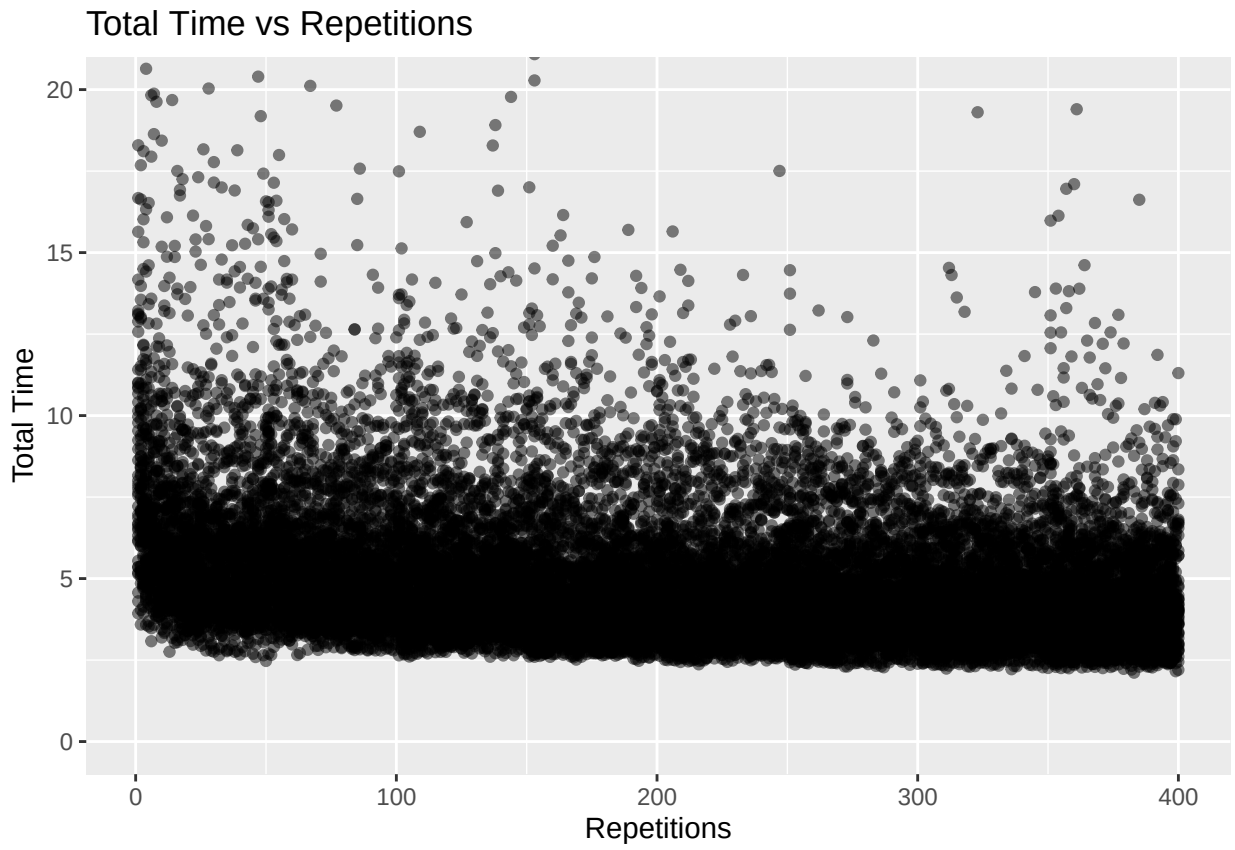
Quantile Regression

```
# Convert 'sessionIndex' and 'rep' to numeric
data$sessionIndex <- as.numeric(data$sessionIndex)
data$rep <- as.numeric(data$rep)

# Create 'repetitions' column to identify which repetition it is
data$repetitions <- (data$sessionIndex - 1) * 50 + data$rep

# Check the updated dataset
head(data)
```

```
## # A tibble: 6 x 39
##   subject sessionIndex   rep H.period DD.period.t UD.period.t    H.t DD.t.i
##   <fct>          <dbl> <dbl>    <dbl>       <dbl>       <dbl>  <dbl>  <dbl>
## 1 s002               1     1    0.149       0.398      0.249  0.107   0.167
## 2 s002               1     2    0.111       0.345      0.234  0.0694  0.128
## 3 s002               1     3    0.133       0.207      0.0744 0.0731  0.129
## 4 s002               1     4    0.129       0.252      0.122  0.106   0.250
## 5 s002               1     5    0.125       0.232      0.107  0.0895  0.168
## 6 s002               1     6    0.139       0.234      0.0949 0.0813  0.130
## # i 31 more variables: UD.t.i <dbl>, H.i <dbl>, DD.i.e <dbl>, UD.i.e <dbl>,
## #   H.e <dbl>, DD.e.five <dbl>, UD.e.five <dbl>, H.five <dbl>,
## #   DD.five.Shift.r <dbl>, UD.five.Shift.r <dbl>, H.Shift.r <dbl>,
## #   DD.Shift.r.o <dbl>, UD.Shift.r.o <dbl>, H.o <dbl>, DD.o.a <dbl>,
## #   UD.o.a <dbl>, H.a <dbl>, DD.a.n <dbl>, UD.a.n <dbl>, H.n <dbl>,
## #   DD.n.l <dbl>, UD.n.l <dbl>, H.l <dbl>, DD.l.Return <dbl>,
## #   UD.l.Return <dbl>, H.Return <dbl>, total_time <dbl>, hold_time <dbl>, ...
```

```
# Create scatter plot with smoothed points and set y-axis limits
ggplot(data, aes(x = repetitions, y = total_time)) +
```

```
  geom_point(alpha = 0.5) +              # Scatter plot with transparency
  coord_cartesian(ylim = c(0, 20)) +     # Set y-axis limits
  labs(title = "Total Time vs Repetitions",
       x = "Repetitions",
       y = "Total Time")
```




```
# Define the quantile levels you are interested in (e.g., 0.25, 0.5, 0.75)
quantile_levels <- c(.025, .25, .5, .75, .975)
```

```
# Function to fit non-linear quantile regression and plot results
fit_and_plot_nonlinear_quantile <- function(dataset, subject_of_interest, show_chart = TRUE, show_predi
  # Filter the data for the specified subject
  filtered_data <- subset(dataset, subject == subject_of_interest)

  # Create an empty data frame to store predictions for repetitions = 401
  predictions_401_df <- data.frame("quantile_level" = numeric(),
                                   "prediction" = numeric(),
                                   stringsAsFactors = FALSE)

  # Create an empty data frame to store predictions
  predictions_df <- data.frame("repetitions" = filtered_data$repetitions)
```

```r
# Define the quantile levels you are interested in
quantile_levels <- c(0.025, 0.25, 0.5, 0.75, 0.975)

# Fit non-linear quantile regression and make predictions for each quantile level
for (quantile_level in quantile_levels) {
  # Define the formula for non-linear quantile regression
  formula <- as.formula(paste("total_time ~ ns(repetitions, df = 3)"))

  # Fit non-linear quantile regression model
  model <- rqss(formula, data = filtered_data, tau = quantile_level)

  # Make predictions for the current quantile level
  predictions <- predict(model, newdata = data.frame(repetitions = filtered_data$repetitions))

  # Store predictions in the data frame under a column with quantile name
  col_name <- paste("quantile", quantile_level * 100, sep = "_")
  col_name <- make.unique(col_name, sep = "_")
  predictions_df[col_name] <- predictions

  # Store predictions in the dataframe for 401 repetitions
  prediction_401 <- predict(model, newdata = data.frame(repetitions = 401))
  predictions_401_df <- rbind(predictions_401_df, data.frame("quantile_level" = quantile_level * 100,
}

# Print predictions for 401 repetitions if show_predictions is TRUE
if (show_predictions) {
  print(predictions_401_df)
}

# Combine the predictions with the original data
combined_data <- cbind(filtered_data, predictions_df)

# Remove duplicate columns (if any)
combined_data <- combined_data[, !duplicated(names(combined_data))]

# Create a scatter plot with non-linear quantile regression lines if show_chart is TRUE
if (show_chart) {
  ggplot(combined_data, aes(x = repetitions, y = total_time)) +
    geom_point(aes(alpha = 0.9)) +  # Scatter plot with transparency
    labs(title = paste("Scatter Plot of Repetitions vs Total Time (Subject:", subject_of_interest, ")
         x = "Repetitions",
         y = "Total Time") +

    # Add non-linear quantile regression lines for all quantile levels
    lapply(quantile_levels, function(q) {
      col_name <- paste("quantile", q * 100, sep = "_")
      geom_line(aes(x = repetitions, y = get(col_name), color = as.factor(q * 100)),
                linewidth = 1)
    }) +

    # Customize plot appearance
    theme_minimal() +
```

```
      # Add legend
      scale_color_manual(name = "Quantile Levels", values = rainbow(length(quantile_levels)),
                         labels = paste0(quantile_levels * 100, "%"))
  }
}
```

Function to fit non-linear quantile regression for each subject and quantile levels of 0.025, 0.25, 0.5, 0.75 and 0.975

```
fit_and_plot_nonlinear_quantile(data, "s036", show_chart = TRUE, show_predictions = TRUE)
```

Executing the function to generate quantile intervals and Predictions for 401 repetitions.

```
##   quantile_level prediction
## 1            2.5   7.190559
## 2           25.0   8.605796
## 3           50.0   9.073603
## 4           75.0  10.430641
## 5           97.5  13.769469
```